

POLITECHNIKA MORSKA W SZCZECINIE

Wydział:

Informatyki i Telekomunikacji

Data pobrania tematu

Data zdania pracy

Katedra:

Informatyki

PRACA DYPLOMOWA INŻYNIERSKA

Dyplomant	Konrad Wargin	Nr albumu: 29291
Specjalność		
Promotor	dr hab. Piotr Borkowski	Ocena:
Recenzent		Ocena:
Egzamin dyplomowy - data		

TEMAT:

**Algorytm dynamicznego generowania scenerii
dostosowującego się do etapu gry oraz wyników gracza**

Dyplomant

Promotor

Dziekan

.....

.....

.....

POLITECHNIKA MORSKA W SZCZECINIE
WYDZIAŁ INFORMATYKI
I TELEKOMUNIKACJI



WYDZIAŁ
INFORMATYKI
I TELEKOMUNIKACJI

Konrad Wargin

**Algorytm dynamicznego generowania scenerii
dostosowującego się do etapu gry oraz wyników gracza**

**Dynamic scene generation algorithm adapting to game
phase and player performance**

Praca inżynierska napisana
w Katedrze Informatyki
pod kierunkiem
dr hab. Piotra Borkowskiego

Szczecin 2025

Oświadczenie

Oświadczam, że praca została sporządzona samodzielnie, tj. poza niezbędnymi konsultacjami nie korzystano z pomocy osób trzecich, a w szczególności nie zlecano opracowania pracy lub jej części innym osobom, jak również wszystkie wykorzystane podczas pisania pracy źródła literaturowe zostały podane do wiadomości.

Jednocześnie oświadczam, że nie korzystano z narzędzi generatywnej sztucznej inteligencji do generowania i analizy danych, sporządzania ilustracji, analizy tekstów źródłowych czy wsparcia redakcyjnego.

Data

Podpis

Oświadczenie

Wyrażam zgodę / nie wyrażam zgody* na udostępnianie mojej pracy w Czytelnii Biblioteki Głównej Politechniki Morskiej w Szczecinie oraz na udostępnianie elektroniczne w sieci BG.

Data

Podpis

* niepotrzebne skreślić

Spis treści

Spis treści	7
Słownik pojęć	9
Wstęp	12
1. Metody projektowania scenerii w grach komputerowych.....	16
1.1. Projektowanie ręczne	16
1.2. Losowa generacja proceduralna.....	17
1.3. Generacja proceduralna wykorzystująca ręcznie utworzone assety.....	20
1.4. Projektowanie ręczne z dopełnianiem poprzez generację proceduralną	22
2. Algorytm proceduralnej generacji scenerii	25
2.1. System punktowy.....	25
2.2. Funkcjonalność proceduralnej generacji scenerii	27
2.3. Funkcjonalność oceny umiejętności gracza.....	30
2.4. Obserwatorzy	33
3. Implementacja autorskiego algorytmu	36
3.1. Wybór silnika oraz języka programowania	36
3.2. Projekt gry wideo	38
3.3. Implementacja systemu punktowego	38
3.4. Implementacja proceduralnej generacji scenerii.....	40
3.5. Konfiguracja i kryteria metody oceniania gracza	46
3.6. Implementacja oceny umiejętności gracza	48
3.7. Implementacja obserwatora przeciwników	51
Zakończenie.....	57
Streszczenie w języku polskim	59
Streszczenie w języku angielskim	61
Bibliografia.....	63
Spis rysunków.....	65
Spis tabel	67
Załącznik 1. Płyta CD z kodem programu	69

Słownik pojęć

Pojęcie	Określenie	
	Język Angielski	Język Polski
Asset	All elements that create game: 3D Models, Sounds, Particle effects and Textures.	„Wszystkie obiekty, które budują grę: Modele 3D, Dźwięki, Efekty cząsteczkowe oraz Tekstury.” [1]
Problem projektowy	Phenomenon or a challenge that shows up during project development, which affects the final experience.	Zjawisko lub wyzwanie pojawiające się w trakcie tworzenia projektu, które wpływa na końcowe doświadczenie.
Minigra	A small additional game that's included in the main game.	Pomniejsza dodatkowa gra zawarta w głównej grze.

Wstęp

Podczas produkcji gier wideo, sposób w który jest tworzona sceneria w grze wpływa na końcowe doświadczenie, które otrzymuje gracz. Z tego powodu projektanci gier wybierają technikę budowania etapów gry, aby dostarczyć konkretne przeżycia, którego doświadczą gracze. Różne metody tworzenia scen w grach przynoszą swoje własne korzyści oraz wady. Z tego powodu, projektant musi dobrać odpowiedni sposób produkcji w celu otrzymania konkretnego efektu.

W wielu projektach typowe metody są bardziej traktowane jako wskazówki i są dostosowywane na potrzeby tworzonej gry. Dzięki temu uzyskuje się unikalne doświadczenie, wyróżniające dany projekt od innych tytułów dostępnych na rynku. Tym samym zachęcając przyszłych klientów do wypróbowania utworzonej produkcji.

Autor w trakcie tworzenia prywatnego projektu, w którym długość rozgrywki miała się nie kończyć, natrafił na problem, który dotyka wiele tytułów tego typu. Częścią tego problemu jest utrata zaangażowania gracza, wynikająca z coraz większej powtarzalności etapów wraz z czasem trwania rozgrywki. Kolejną częścią problemu jest utrzymanie odpowiedniego balansu poziomu trudności, tak aby utrzymać zaangażowanie i odpowiednie poczucie wyzwania dla gracza.

Wiedząc, jak ważna dla graczy jest dostępność zróżnicowanych poziomów trudności w produkcjach, pozwalająca na personalizację otrzymywanego doświadczenia, autor doszedł do wniosku, że w celu utrzymania jak największego zaangażowania, gra potrzebuje stale dostarczać odpowiedni stopień wyzwania. Większość produkcji na rynku posiada zdefiniowane poziomy trudności. Jednakże to podejście nie zawsze się sprawdza w grach, w których rozgrywka ma trwać nieskończenie długo. W pewnym momencie, niezależnie od wybranej wielkości wyzwania, gracz trafi na moment, który będzie powodować znudzenie, bądź oferowane wyzwanie stanie się niemożliwe do pokonania.

W celu rozwiązania powyższego problemu, zapoczątkowana została idea algorytmu generacji poziomów, dostosowująca generowane poziomy unikalnie dla każdego gracza poprzez odpowiednią manipulację poziomem trudności. Autor pracy wydedukował, że w celu otrzymania niekończącej się rozgrywki, które utrzyma zainteresowanie gracza, należy dostosować w kolejnych etapach generowane wyzwanie. Porzucając tym samym mechanizm zwiększania poziomu trudności w stałym tempie.

Celem niniejszej pracy jest zaprojektowanie i implementacja algorytmu generacji scenerii, dostosowującego się do etapu gry oraz wyników gracza. Algorytm wykorzystuje metodę tworzenia scenerii, poprzez ręczne projektowanie z dopełnianiem sceny. Zamysłem algorytmu jest utworzenie unikalnego doświadczenia dla gracza. Ma on dostarczyć stale satysfakcjonujące wyzwanie osobom o wysokich umiejętnościach. Równocześnie ułatwiać

wygenerowane etapy graczom o słabszych zdolnościach, w celu utrzymania stałego zaangażowania.

W ramach pierwszego rozdziału zostaną przedstawione popularne metody projektowania scenerii w grach komputerowych. Opisany zostanie ich zamysł działania, przynieszone przez nie wady i zalety dla tworzonego projektu. Zaprezentowane zostaną także przykładowe produkcje, w których podane metody zostały wykorzystane.

Drugi rozdział zdekonstruuje i przedstawi zamysł działania zaprojektowanego uniwersalnego algorytmu dynamicznej generacji scenerii tworzonego w ramach tej pracy. Wskaże również jak przedstawiony algorytm może zostać dostosowany na własne potrzeby. Przedstawi również potencjalny wpływ, jakie wybrane metody implementacji przynoszą efekty w ramach doświadczenia końcowego, które otrzyma gracz w trakcie rozgrywki.

W trzecim rozdziale zostanie zaprezentowana implementacja przykładowej wersji algorytmu przedstawionego w rozdziale drugim. Dokonano tego poprzez integrację z projektem prostej gry platformowej utworzonej za pomocą silnika Unreal Engine 5.3.2. Zintegrowany w ten sposób algorytm będzie generował etapy, dostosowując poziom trudności poprzez ocenę czynników w trakcie rozgrywki gracza takich jak: czas trwania walki oraz ilość otrzymywanych obrażeń. W ten sposób zostanie przedstawione potencjalne praktyczne zastosowanie oraz możliwości, które oferuje algorytm będący celem niniejszej pracy.

1. Metody projektowania scenerii w grach komputerowych

„Projektant gry jest jak architekt kreujący dom – dba o jego funkcjonalność, odpowiednie standardy i dopasowanie materiałów do wykonania. Rolą takiej osoby jest łączenie elementów rozgrywki w spójną całość, poczynając od poziomów, przez minigry, na znajdkach kończąc.” [1]

Sposób w jaki jest tworzona sceneria w grze ma wpływ na całokształt projektu oraz końcowe doświadczenie, które będzie dostarczane graczowi. Projektanci gier muszą być świadomi, że sposób w jaki są tworzone elementy gry, wpływają na całokształt projektu a także na potencjalne przeszkody lub możliwości, które owe metody przynoszą. W zależności od potrzeb projektu oraz docelowej wizji projektanta konieczne jest dobranie odpowiedniego sposobu tworzenia poszczególnych elementów gry, zwracając uwagę, aby współgrały z innymi elementami projektu.

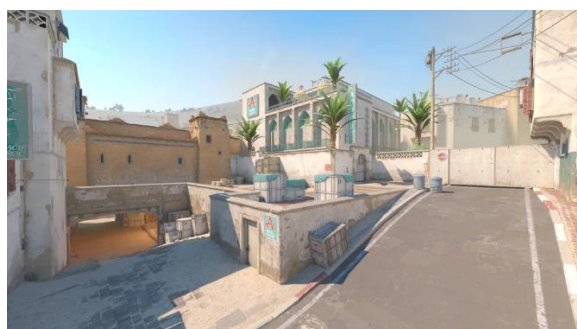
Ten rozdział przedstawi zatem bazowe metody projektowania scenerii w grach, ich wykorzystanie, a także wpływ jaki mają na projekt.

1.1. Projektowanie ręczne

Projektowanie ręczne to metoda oparta na tworzeniu każdej sceny oraz ich elementów w grze od zera. Nie wykorzystuje się w tym projektowaniu elementów losowości z wyjątkiem nielicznych przypadków na potrzeby etapu gry. Ta metoda wybierana jest wówczas, gdy projektant pragnie wytworzyć specyficzne doświadczenie w określonym momencie, w podobny sposób jak w innych mediach rozrywkowych takich jak: książki, filmy itp.

Powyższa metoda najczęściej wykorzystywana jest w grach, które mają opowiadać niepowtarzalną historię. Najczęściej scenerie są tworzone ręcznie w grach przygodowych, logicznych oraz opartych na rywalizacji z innymi graczami.

Jednym z najbardziej znanych przykładów scen zaprojektowanych ręcznie jest mapa „Dust II” z serii gier „Counter-Strike” (Valve, 2000~2012), będąca najbardziej rozpoznawaną scenerią przez fanów serii, której fragment został ukazany w poniższej grafice (rys. 1.1.).



Rysunek 1.1. Zrzut ekranu sceny "Dust II" z gry "Counter-Strike 2"
Źródło: [3]

Kolejnym doskonałym przykładem gry wykorzystującej projektowanie ręczne scenerii jest gra „GRIS” (Nomada Studio, Conrad Roset, 2018), w której każda scena została namalowana ręcznie tworząc doświadczenie rodem z wystawy artysty w galerii sztuki. Jedną z takich scenerii ukazuje poniższy zrzut ekranu (rys. 1.2.), wykorzystywany jako materiał promocyjny gry.



Rysunek 1.2. Zrzut ekranu z gry "GRIS"
Źródło: [4]

Wady i zalety projektowania ręcznego

Zaletami tego typu podejścia jest:

- pełna kontrola nad dostarczonym doświadczeniem,
- łatwiejsze wprowadzanie poprawek oraz dokonywanie zmian.

Natomiast do wad tego rodzaju projektowania zaliczamy:

- większy nakład pracy dla całego zespołu,
- większe potencjalne straty w sytuacji, gdy element nie zostanie wykorzystany lub będzie tworzył problemy.

1.2. Losowa generacja proceduralna

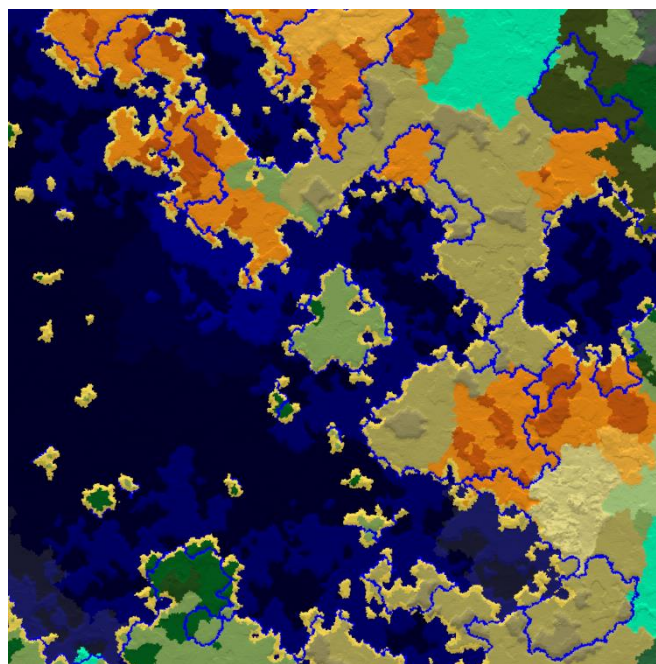
Generacja proceduralna to technika generowania zawartości poprzez wykorzystywanie algorytmów w celu wytwarzania nowej zawartości zamiast projektowania

ręcznego. Podczas tworzenia gier może być wykorzystywana do generacji losowej różnych elementów takich jak zagadek, przeszkód, nagród a nawet całych scen.

Tworząc sceny w pełni za pomocą generacji proceduralnej mamy pewność, że każda stworzona scena będzie unikalna. Projektując algorytmy generacji proceduralnej, trzeba zaimplementować je w taki sposób, aby zwracana zawartość była jak najbardziej zbliżona do oczekiwanego efektu.

Najczęściej ta metoda jest wykorzystywana w grach gatunkowo określonych jako Piaskownica (z ang. „Sandbox”) oraz w grach, które dają graczowi większe możliwości interakcji z terenem.

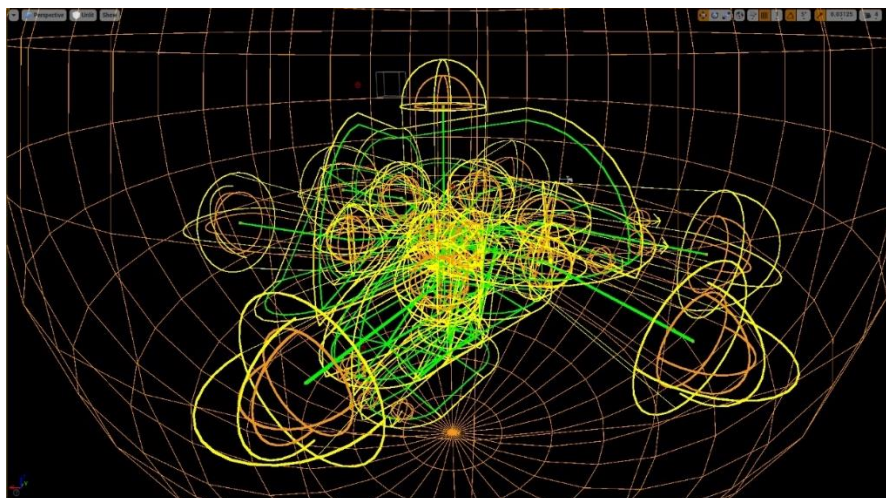
Najbardziej znanym tytułem, który generuje scenę w pełni za pomocą generacji proceduralnej jest „Minecraft” (Mojang Studios, Markus Persson, 2011), w której scena jest generowana losowo przy rozpoczęciu każdej nowej rozgrywki. Sposób działania algorytmu generacji świata w tym tytule został dokładnie opisany w artykule „The World Generation of Minecraft” napisanego przez Alana Zucconiego [5]. Poniższa grafika (rys. 1.3.) przedstawia przykład generacji terenu tworzego za pomocą wyżej wspomnianego algorytmu.



Rysunek 1.3. Przykładowa generacja terenu o wielkości 2048 bloków w grze "Minecraft".
Źródło: [5]

Kolejnym przykładem wykorzystania generacji proceduralnej do generacji sceny jest „Deep Rock Galactic” (Ghost Ship Games, Mikkel Martin Pedersen, 2020) w której każdy etap posiada system losowo generowanych jaskiń gdzie odbywa się akcja gry. Mechanika niszczenia terenu pozwala graczom tworzyć własne ścieżki pozbywając się problemu generacji sceny, w której jest niemożliwe ukończenie etapu. Fragmenty procesu generacji

przykładowej sceny, wykorzystującej powyższy system, zostały ukazane w rysunkach poniżej (rys. 1.4., rys. 1.5.).



Rysunek 1.4. Wzór wygenerowanej jaskinii przed wygenerowaniem elementów graficznych w Deep Rock Galactic z poziomu edytora w silniku Unreal Engine 4
Źródło: [6]



Rysunek 1.5. Wygenerowana jaskinia z rysunku 1.4
Źródło: [6]

Wady i zalety losowej generacji proceduralnej

Zaletami tej metody są:

- każda generacja scenerii oferuje unikalne rezultaty,
- nieskończona regrywalność poprzez oferowanie kompletnie nowego doświadczenia z każdą rozgrywką.

Wady losowej generacji proceduralnej:

- trudność w utworzeniu sprawnie działającego algorytmu generacji,
- algorytm może wygenerować nieprzewidziane lub nieodpowiednie wyniki,
- minimalne zmiany posiadają wielki wpływ na końcowe efekty generacji.

1.3. Generacja proceduralna wykorzystująca ręcznie utworzone assety

Algorytm generacji proceduralnej zamiast budowania elementów od zera wykorzystuje wcześniej przygotowane assety, które zostały zaprojektowane ręcznie, łącząc je w spójną całość, tym samym tworząc nową scenę.

To podejście daje projektantowi większą kontrolę nad uzyskanym efektem zwrotnym algorytmu proceduralnej generacji. Dzięki oznaczaniu elementów poprzez metadane, daje możliwość projektantowi ustalania zasad działania generacji poprzez deklarowanie wymaganych segmentów zawartych w scenie. Segmentacja elementów scenerii oferuje możliwość kontroli sposobu w jaki elementy sceny są łączone ze sobą, tak aby każda lokacja na mapie była dostępna dla gracza.

Ilość przygotowanych segmentów scenerii wpływa na końcową ilość możliwych wariantów generacji mapy.

Najczęściej ta metoda projektowania scenerii jest wykorzystywana w grach opartych na mechanice losowości, głównie w grach z gatunku Roguelike¹.

Przykładem gry wykorzystującą generację proceduralną z użyciem ręcznie utworzonych assetów jest seria gier „The Binding of Isaac” Edmunda McMillena z 2011 roku. Algorytm generacji sceny zaimplementowany w tych grach stał się sporą inspiracją dla wielu deweloperów i jest on wciąż powszechnie adaptowany na potrzeby wielu dzisiejszych produkcji z gatunku Roguelike. Sposób działania algorytmu generacji w „The Binding of Isaac” został poddany analizie i zamieszczony w artykule „Dungeon Generation in Binding of Isaac” przez programistę o pseudonimie „BorisTheBrave” w blogu internetowym „BorisTheBrave.com” [7]. Poniższa grafika (rys. 1.6.) prezentuje uproszczoną formę przykładowej scenerii wygenerowanej za pomocą powyższego algorytmu.

¹ Roguelike to gatunek gier charakteryzujący się proceduralnie generowanymi mapami, turową rozgrywką oraz permanentną śmiercią postaci gracza. Gatunek został zapoczątkowany i spopularyzowany poprzez grę „Rogue” Micheala Toy-a oraz Glenna Wichmana z 1980 roku.



Rysunek 1.6. Przykładowa generacja mapy z gry "The Binding of Isaac: Rebirth". Każdy blok reprezentuje osobny wcześniej przygotowany fragment scenerii tworzący razem całą scenę.
Źródło: [7]

Wady i zalety generacji proceduralnej z wykorzystaniem ręcznie utworzonych assetów

Generacja proceduralna z wykorzystaniem ręcznie utworzonych assetów posiada następujące zalety:

- pewność w zakresie wygenerowanej zawartości poprzez ustalone zasady generacji,
- łatwość połączenia poszczególnych elementów scenerii bez wystąpienia artefaktów,
- łatwość modyfikacji docelowego rezultatu,
- łatwość modyfikacji wykorzystywanych segmentów przy wykryciu błędu lub wprowadzania zmian,
- liczba możliwych permutacji wygenerowanych scen zwiększa się z ilością dostępnych segmentów,
- prosta implementacja algorytmu generacji proceduralnej,

Metoda posiada jednak również wady:

- powtarzalność i potencjalne znużenie w doświadczeniu gracza poprzez wtórną generację wcześniej wykorzystanych segmentów,
- wadliwa implementacja może tworzyć niemożliwe rozwiązania poprzez blokadę dostępu do wygenerowanych segmentów mapy.

1.4. Projektowanie ręczne z dopełnianiem poprzez generację proceduralną

Odwrótnym podejściem do poprzedniej metody jest wykorzystywanie sceny zaprojektowanych ręcznie, które są następnie uzupełniane detalami w sposób losowy. Takimi detalami mogą być między innymi: dekoracje, interaktywne obiekty, nagrody oraz wszelkiego rodzaju przeszkody.

Wykorzystanie generacji proceduralnej w ten sposób pozwala projektantom gry na dodanie większego zróżnicowania scenerii, zachowując pełną kontrolę nad najważniejszymi elementami mapy. Odpowiednio wykorzystane może również dodać rozgrywce więcej elementów zaskoczenia oraz zachęcić gracza do adaptacji zależnie od aktualnej sytuacji podczas gry.

Przykładem tytułu wykorzystującego powyższy sposób projektowania scenerii jest niedostępny już „The Cycle Frontier” (Yager, Matt Lightfoot, 2022) z gatunku strzelanek ewakuacyjnych. The Cycle zawierało trzy dostępne ręcznie zaprojektowane mapy, które były uzupełniane w czasie rzeczywistym poprzez generację proceduralną o różnych przeciwników, przedmioty które gracz mógł zbierać, jak również i losowe wydarzenia w zależności od lokalizacji na scenie, jak ukazano na rysunku poniżej (rys. 1.7.).



Rysunek 1.7. Mapa "Bright Sands" z gry "The Cycle Frontier" z oznaczeniem kolorystycznym stref wpływających na wyniki generacji losowej.

Źródło: [8]

Tworzony algorytm będący celem tej pracy bazuje na powyżej opisanej metodzie projektowania scenarii.

Wady i zalety projektowania ręcznego z dopełnianiem poprzez generację proceduralną

Wady i zalety powyższej metody są identyczne jak przy metodzie projektowania ręcznego, z natury wymogu użycia wspomnianej metody w trakcie pracy. Jednak posiada zauważalne różnice:

- oferuje większą regrywalność w porównaniu z projektowaniem ręcznym,
- posiada większe ryzyko wystąpienia błędów w wyniku zastosowania generacji losowej.

2. Algorytm proceduralnej generacji scenarii

Zamysłem całego algorytmu przedstawionego w tej pracy jest proceduralna generacja scenarii oferująca stale zwiększający się poziom trudności, adaptujący się do aktualnych wyników gracza, w celu dostosowania skalowania stopnia trudności poprzez obserwację zmiennych zadeklarowanych przez projektanta gry podczas rozgrywki gracza i oceny gracza na podstawie zebranych danych.

Dzięki tej obserwacji, algorytm otrzymuje informacje, czy skalowanie poziomu trudności powinno zostać zmniejszone w wypadku gorszych wyników gracza lub zwiększone w przypadku gdy graczowi idzie lepiej niż przewidywano. W ten sposób algorytm generuje etapy, które zadowolą gracza niezależnie od jego umiejętności.

Dzięki temu algorytm reguluje dostarczany poziom trudności gry aby utrzymać zadowolenie gracza bez powodowania frustracji przez zbyt wysoki poziom trudności lub potencjalne znużenie, kiedy gra jest zbyt łatwa w porównaniu do zdolności gracza, jak ukazano na grafice poniżej (rys. 2.1.).



Rysunek 2.1. Krzywa obrazująca doświadczanie gry przez gracza w projektowaniu gier
Źródło: [9]

2.1. System punktowy

Działanie generacji proceduralnej w algorytmie zawartym w pracy jest inspirowane poprzez systemy punktowe używane w grach bitewnych takich jak „Warhammer 40.000” (Games Workshop, 1987) czy „Władca Pierścieni: Strategiczna Gra Bitewna” (Games Workshop, 2001). W tego typu grach, przed każdą rozgrywką gracze ustalają dostępną pulę

punktów, którą potem wydają na dowolne figurki reprezentujące jednostki. Każda jednostka posiada swój własny koszt punktowy oraz właściwości. Gracz „kupując” w ten sposób jednostki tworzy własną armię, którą używa podczas meczu. Zainteresowanie tego typu grami pokazuje między innymi poniższe zdjęcie (rys. 2.2.), które zarejestrowało prezentację gry Warhammer 40.000 podczas konwentu Operation Sci-Fi Con 2015.



Rysunek 2.2. Zdjęcie przedstawiające prezentację gry Warhammer 40.000 w Operation Sci-Fi Con 2015, w Niemczech z 2015 roku.

Źródło: [10]

Pula punktowa

Pula punktowa jest zmienną liczbową w algorytmie określającą ilość dostępnych „punktów” do wydania przez algorytm generacji proceduralnej w trakcie generowania sceny.

Wartość puli punktowej jest wprowadzana manualnie w sposób arbitralny przez osobę implementującą algorytm oraz może być później modyfikowana poprzez działanie algorytmu.

Z założenia, im wartość puli punktowej jest większa tym większy będzie poziom trudności wygenerowanego etapu.

Koszt punktowy

W celu odpowiedniego działania algorytmu, każdy element, który może zostać wygenerowany podczas generacji poziomu, musi zawierać metadane określające jak dużo musi zostać wydanych punktów z puli punktowej aby obiekt został wygenerowany.

Wartości kosztów punktów dla określonych elementów są wprowadzane manualnie w sposób arbitralny przez osobę implementującą algorytm oraz mogą być później modyfikowane poprzez działanie algorytmu.

Z założenia, im większa wartość kosztu obiektu, tym jest traktowany jako „trudniejszy” i będzie rzadziej generowany.

W wypadkach kiedy koszt punktowy obiektu przekracza aktualną pozostałą wartość puli punktowej, nie może zostać wygenerowany.

2.2. Funkcjonalność proceduralnej generacji scenarii

Główną funkcjonalnością tworzonego algorytmu jest proceduralna generacja scenarii. Ze względu na różnice wynikające z używanych narzędzi, tworzonej bądź istniejącej architektury w projektach oraz zależnie od potrzeb projektanta, poniższa funkcjonalność wymaga odpowiedniego dostosowania, w celu integracji z projektem.

Z tego powodu, schemat działania który zostanie przedstawiony poniżej powinien być traktowany jako szablon, wymagający modyfikacji, w celu uzyskania oczekiwanych wyników końcowych.

Pokazana baza funkcji proceduralnej generacji scenarii działa z poniższymi założeniami:

- algorytm zawiera Pulę punktową przedstawioną w podrozdziale 2.1.1,
- scenaria jest uzupełniania proceduralnie w wyznaczonych miejscach do generacji przez wybrane wcześniej obiekty, tworzące pulę obiektów do generacji,
- każdy obiekt, który może zostać wygenerowany, posiada przypisany koszt punktowy przedstawiony w podrozdziale 2.1.2,
- algorytm uzupełnia scenarię proceduralnie poprzez generację losowo wybranych obiektów z puli obiektów do generacji do momentu, aż zabraknie dostępnych miejsc do generacji, bądź pula punktowa zostanie wyczerpana.

Funkcja generacji proceduralnej scenarii z przedstawionymi założeniami działa w następujący sposób:

W celu poprawnego działania, funkcja wymaga dostępu do informacji o wyznaczonych wcześniej miejscach do generacji na scenie oraz wymaga wskazania które obiekty mogą zostać wygenerowane. Obiekty generowane na scenie tworzą zbiór obiektów, oznaczony jako pula obiektów do generacji.

Podczas uruchomienia funkcji tworzona jest zmienna tymczasowa odpowiedzialna za przechowywanie pozostałej ilości dostępnych punktów używanych do generacji. Wartość tej zmiennej jest równa wartości puli punktów z podrozdziału 2.1.1.

Następnie wykonywana jest poniższa pętla instrukcji, do momentu, aż nie skończą się dostępne miejsca do generacji, bądź pozostałe punkty nie będą mniejsze lub równe 0.

Na początku każdej iteracji, algorytm wybiera losowo obiekt z dostępnej puli obiektów do generacji. Aby wylosowany obiekt mógł zostać wygenerowany musi on spełniać dwa warunki:

1. Przypisana wartość kosztu punktowego wylosowanego obiektu musi być mniejsza bądź równa wartości pozostałych punktów,
2. Wygenerowanie wylosowanego obiektu musi pozwolić na pełne wykorzystanie pozostałych punktów w kolejnych pętlach funkcji generacji.

Jeżeli wylosowany obiekt spełnia oba warunki, algorytm wybiera losowo jeden z wyznaczonych dostępnych miejsc do generacji, po czym generuje na scenie tak wylosowany obiekt w wyznaczonym miejscu. Po wygenerowaniu obiektu w ten sposób, od zmiennej przechowującej pozostałe punkty zostaje odejmowana wartość kosztu punktowego wygenerowanego obiektu.

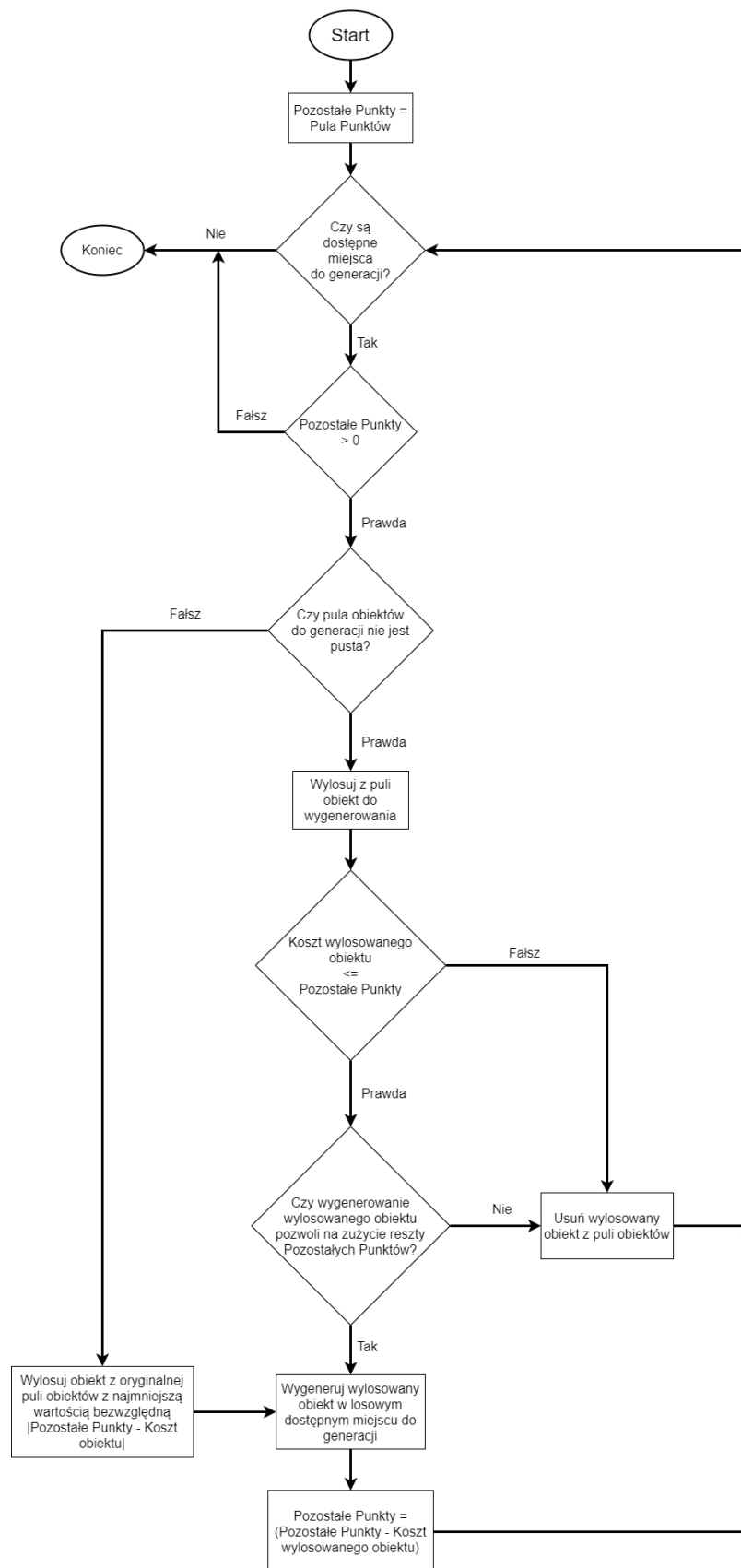
W sytuacji gdy wylosowany obiekt nie spełnia powyższych warunków, jest on usuwany z puli dostępnych obiektów w celu uniknięcia sytuacji, gdzie algorytm ponownie wylosuje ten sam obiekt.

Po sukcesywnej generacji bądź usunięciu z puli wylosowanego obiektu, rozpoczyna się następna iteracja powyższej pętli instrukcji.

Po wystarczającej ilości iteracji, może nastąpić sytuacja, gdzie pula obiektów do generacji stanie się pusta, ponieważ żaden wyznaczony obiekt nie będzie spełniał obu wcześniej przedstawionych warunków. W takim wypadku algorytm wybiera losowy obiekt z oryginalnej puli obiektów do generacji, po czym kontynuuje wykonywanie pętli.

W celu większej dokładności, powinien zostać wylosowany element, dla którego wartość bezwzględna różnicy pozostałych punktów od przypisanego kosztu punktowego obiektu będzie jak najmniejsza.

Działanie powyższej funkcjonalności zostało ukazane na schemacie blokowym poniżej (rys. 2.3.).



Rysunek 2.3. Schemat opisujący metodę działania funkcjonalności generacji proceduralnej scenarii

Źródło: Opracowanie własne

2.3. Funkcjonalność oceny umiejętności gracza

W celu dostosowywania wyników generacji do umiejętności gracza, algorytm wymaga posiadania funkcjonalności, która będzie zbierała zadeklarowane przez projektanta gry dane z rozgrywki oraz odpowiednio je oceniała w celu ustalenia umiejętności gracza oraz potencjalnej modyfikacji puli punktowej.

Projektant gry deklaruje jakie zmienne powinny być badane oraz w jaki sposób będą one oceniane, aby uzyskać zamierzony efekt działania całości algorytmu.

Konfiguracja funkcjonalności oceny wymaga sporej precyzji, ponieważ sposób oceniania oraz minimalne zmiany w zmiennych w algorytmie, posiadają drastyczny wpływ na końcowy wynik generacji sceny.

Dane kontrolne

W zależności jaki zostanie zastosowany zakres i źródło danych kontrolnych wykorzystywanych przy ocenie gracza, można otrzymać inne efekty, jak wykazano w tabeli poniżej (tab. 2.1.):

Tabela 2.1. Typy danych kontrolnych dla algorytmu oceny umiejętności gracza

Typ danych	Pochodzenie	Zalety	Wady
Statyczna średnia	Statyczny docelowy oczekiwany wynik ustalany ręcznie przez projektanta gry.	- Pełna kontrola nad wynikiem interpretacji wyników gracza, - Łatwość implementacji, - Identyczne doświadczenie dla różnych graczy.	- Brak precyzyjnego dostosowania poziomu trudności do zdolności gracza, - Ryzyko manipulacji wyniku generacji przez gracza.

Typ danych	Pochodzenie	Zalety	Wady
Początkowa kalibracja	Wyniki gracza z aktualnego podejścia są zbierane na początku rozgrywki i używane jako dane kontrolne do oceny dalszej rozgrywki gracza.	- Początek gry oferuje identyczne doświadczenie dla każdego gracza, - Stały punkt odniesienia oczekiwanej średniej po zakończeniu kalibracji.	- Wysokie ryzyko manipulacji wyniku generacji przez gracza podczas fazy kalibracji, - Brak dostosowania średniej do późniejszych wyników gracza, - Ryzyko nieprawidłowej oceny gracza.
Ocena na bazie aktualnego przebiegu	Wyniki gracza z aktualnego podejścia są ciągle zbierane i używane jako dane kontrolne do oceny rozgrywki gracza.	- Początek gry oferuje identyczne doświadczenie dla każdego gracza, - Oczekiwana średnia jest stale dostosowywana z czasem trwania gry, - Małe ryzyko manipulacji wyników generacji.	- Możliwość zaburzenia oczekiwanej średniej przez pojedyncze dane wychodzące poza normę, - Wpływ nowych danych zmniejsza się z wraz zebraną ich ilością.

Typ danych	Pochodzenie	Zalety	Wady
Ocena na bazie wszystkich przebiegów	Wyniki gracza z każdego podejścia są zbierane i używane jako dane kontrolne do oceny rozgrywki gracza.	- Unikalne doświadczenie dla każdego gracza.	- Wpływ nowych danych zmniejsza się z wraz zebraną ich ilością, - Drastyczne zmiany poziomu trudności przy małych różnicach umiejętności gracza, - Możliwość utworzenia nieprzyjemnego doświadczenia przez wyniki poprzednich rozgrywek, - Zwiększone użycie pamięci oraz zasobów urządzenia.
Ocena na bazie aktualnego przebiegu z limitem pamięci	Określona ilość najnowszych wyników gracza z aktualnego podejścia jest zbierana i używana jako dane kontrolne do oceny rozgrywki gracza.	- Początek gry oferuje identyczne doświadczenie dla każdego gracza, - Oczekiwana średnia jest stale dostosowywana z czasem trwania gry oraz ze zmianami w umiejętnościach gracza.	- Możliwość zaburzenia oczekiwanej średniej przez pojedyncze dane wychodzące poza normę, - Ryzyko manipulacji wyniku generacji przez gracza.

Typ danych	Pochodzenie	Zalety	Wady
Ocena na bazie wyników wszystkich graczy	Wyniki graczy z całego świata są zbierane i używane jako dane kontrolne do oceny rozgrywki aktualnego gracza.	- Oferuje identyczne stale zmieniające się doświadczenie dla wszystkich graczy, - Utrzymuje zaangażowanie graczy do ich zwiększających się umiejętności.	- Wymóg utworzenia i utrzymywania serwera przechowującego dane wyników graczy, - Wymuszenie stałego połączenia z siecią dla gracza, - Wraz ze zwiększającą się globalną średnią zwiększa się próg wejścia dla nowych graczy.

Wpływ algorytmu oceny na generację

Poprzez wynik algorytmu oceny gracza można manipulować wartością puli punktowej zarówno jak i kosztem pojedynczych obiektów.

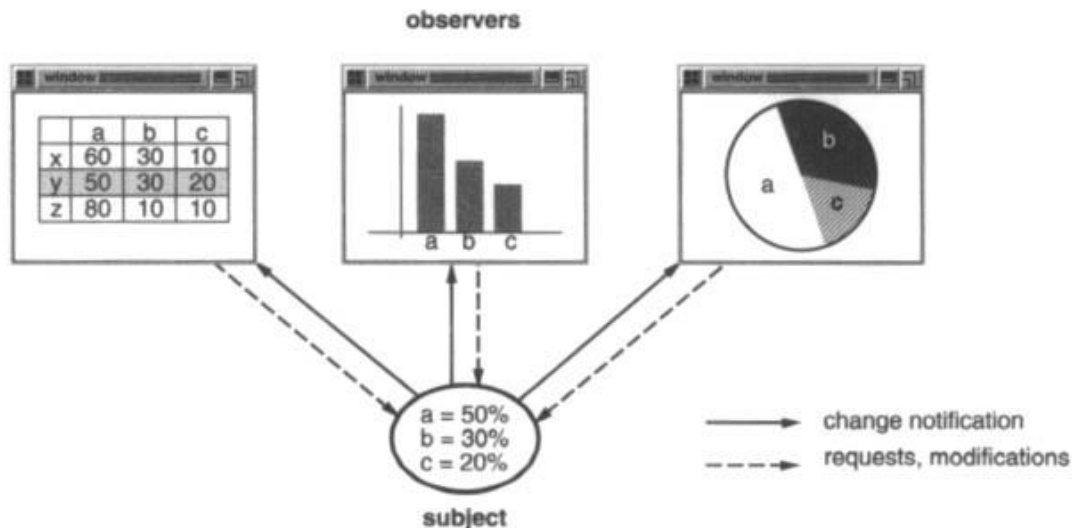
W przypadku modyfikacji puli punktowej, dostosowuje się ogólny przewidywany poziom trudności wygenerowanej sceny.

Można również zaprogramować algorytm, tak aby wpływał na osobne koszty wygenerowanych obiektów z aktualnego etapu, w celu zmniejszenia bądź zwiększenia prawdopodobieństwo wygenerowania danego assetu. Wymaga to jednak rozdzielenia ogólnej oceny gracza dodatkowo na osobną ocenę każdego obiektu.

2.4. Obserwatorzy

W celu zbierania wymaganych danych do oceny, potrzebne są osobne algorytmy zintegrowane z obiektami związanymi z badanymi parametrami oceny, zwane obserwatorami.

Obserwatorzy są powszechnym schematem projektowym wykorzystywanym w programowaniu, którego rolą jest tworzenie relacji jeden do wielu, informującym powiązane obiekty o zmianie stanu jednego obiektu, w celu zaktualizowania stanu powiązanych elementów, jak w rysunku poniżej (rys. 2.4.) [2].



Rysunek 2.4. Relacja obserwatora jeden do wielu
Źródło: [2]

Schemat programowania obserwatorów jest domyślnie wbudowany w wielu językach programowania jak na przykład: Java w bibliotece „java.util.Observer”, C# w komendzie „event” oraz w wizualnym języku skryptowym Blueprint w silniku Unreal Engine poprzez funkcjonalność „Event Dispatcher”.

Celem obserwatorów w przedstawianym algorytmie będącym tematem pracy jest zbieranie danych wymaganych w ramach oceny gracza i przekazywanie je funkcjonalności odpowiedzialnej za wykorzystanie zebranych danych do kalkulacji oceny gracza, opisanego w podrozdziale 2.3.

3. Implementacja autorskiego algorytmu

W poniższym rozdziale zostanie zaprezentowana implementacja autorskiego algorytmu dynamicznej generacji scenarii przedstawionej w rozdziale drugim. Początek rozdziału krótko omawia wykorzystane narzędzia oraz projekt gry wideo, do którego algorytm będący celem pracy został dostosowany oraz zaimplementowany. Dalsza część rozdziału zawiera konfigurację oraz dokładną implementację kluczowych elementów algorytmu przedstawionego w rozdziale drugim.

3.1. Wybór silnika oraz języka programowania

Do wykonania algorytmu został zastosowany silnik Unreal Engine 5.3.2 oraz jego wbudowany język skryptowy Blueprint, z uwagi na jego znajomość z wcześniejszych projektów wykorzystujących powyższy silnik.

Unreal Engine 5.3.2

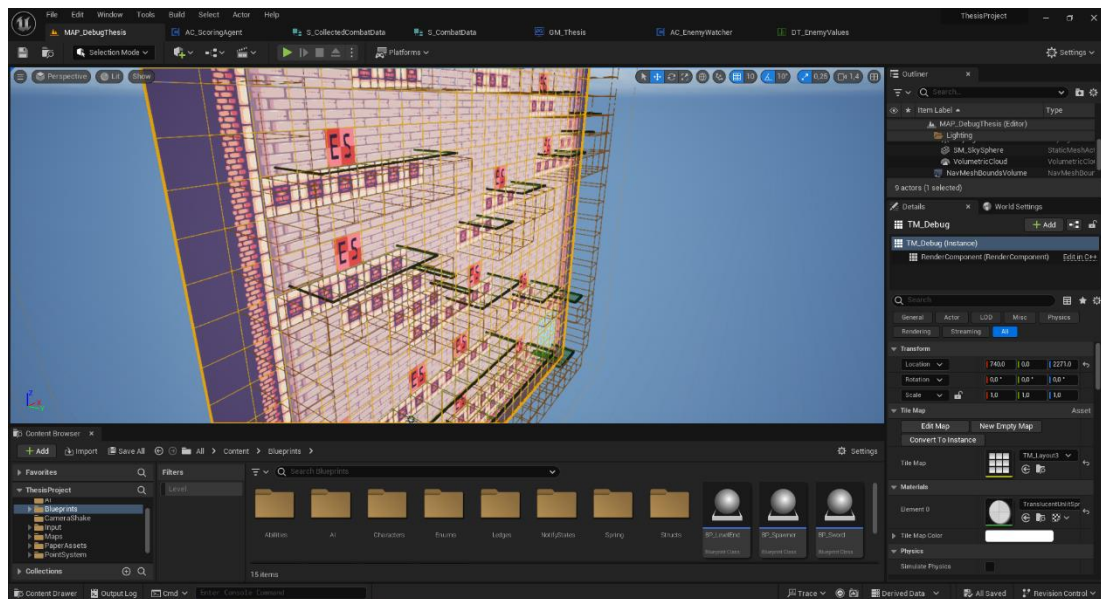
Unreal Engine to silnik graficzny 3D umożliwiający tworzenie gier komputerowych, modelowanie architektury oraz produkowanie animacji oraz filmów. Silnik został stworzony przez Tima Sweeney-a w studiu Epic Games. Pierwsza wersja silnika została wydana w 1998 roku. Najnowsza główna wersja Unreal Engine 5, która jest wykorzystywana w ramach projektu będącego celem tej pracy, została wydana w 2022 roku.

Unreal Engine umożliwia programowanie w językach:

- C++,
- Wizualny język skryptowy Blueprint,
- Python.

W momencie pisania niniejszej pracy, Unreal Engine jest jednym z liderów silników graficznych w branży gier.

Poniższy zrzut ekranu (rys. 3.1.) przedstawia główny interfejs edytora Unreal Engine.



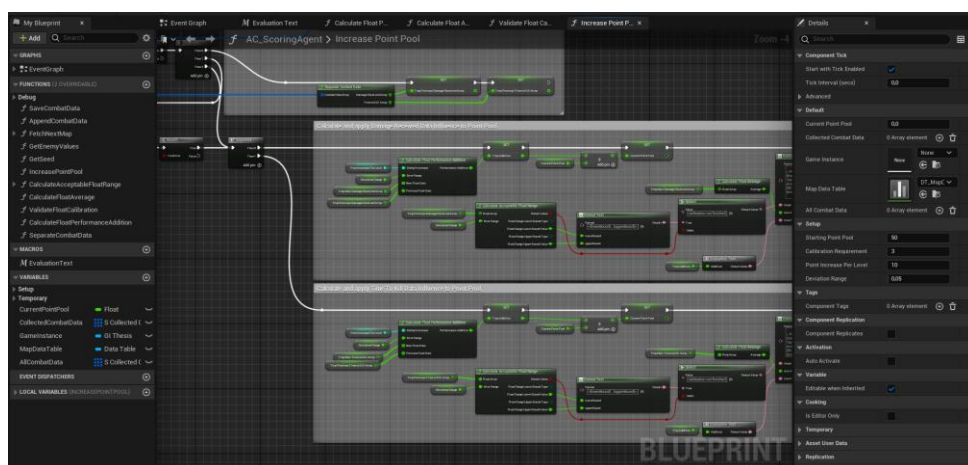
Rysunek 3.1. Edytor silnika Unreal Engine 5.3.2

Źródło: Opracowanie własne

Wizualny język skryptowy Blueprint

Blueprint to autorski wizualny język skryptowy stworzony na potrzeby Unreal Engine przez Epic Games, pozwalający w sposób wizualny tworzyć klasy, funkcje oraz zmienne w projekcie Unreal Engine z poziomu edytora. Jako bazę do klas Blueprint wykorzystywane są klasy z języka C++ [11].

Język skryptowy Blueprint pozwala osobom bez znajomości kodowania na programowanie funkcjonalnego kodu, zarówno jak nadaje się do szybkiego programowania mało skomplikowanych klas a także prototypowania gier w krótkim czasie, za pomocą edytora ukazanego na poniższym zrzucie ekranu (rys. 3.2.).



Rysunek 3.2. Edytor Blueprint w edytorze Unreal Engine 5.3.2

Źródło: Opracowanie własne

3.2. Projekt gry wideo

Algorytm będący tematem pracy został zintegrowany z prostym projektem gry wideo utworzonym w ramach kursu „The Ultimate Unreal Engine 2D Game Development Course” [12] z serwisu kursów internetowych Udemy.

Celem gracza w grze widocznej poniżej (rys. 3.3.), jest pokonanie wszystkich przeciwników w aktualnym poziomie aby odblokować przejście do kolejnego etapu, próbując przejść jak największą ilość etapów o zwiększających się poziomach trudności.



Rysunek 3.3. Zrzut ekranu z gry używanej w projekcie
Źródło: Opracowanie własne

3.3. Implementacja systemu punktowego

Pula punktowa

System punktowy w algorytmie wykorzystuje pulę punktową zarządzaną w klasie `AC_ScoringAgent`, przechowywaną w zmiennej `float` „`CurrentPointPool`”, z ustaloną wartością startową deklarowaną w zmiennej `integer` „`StartingPointPool`”.

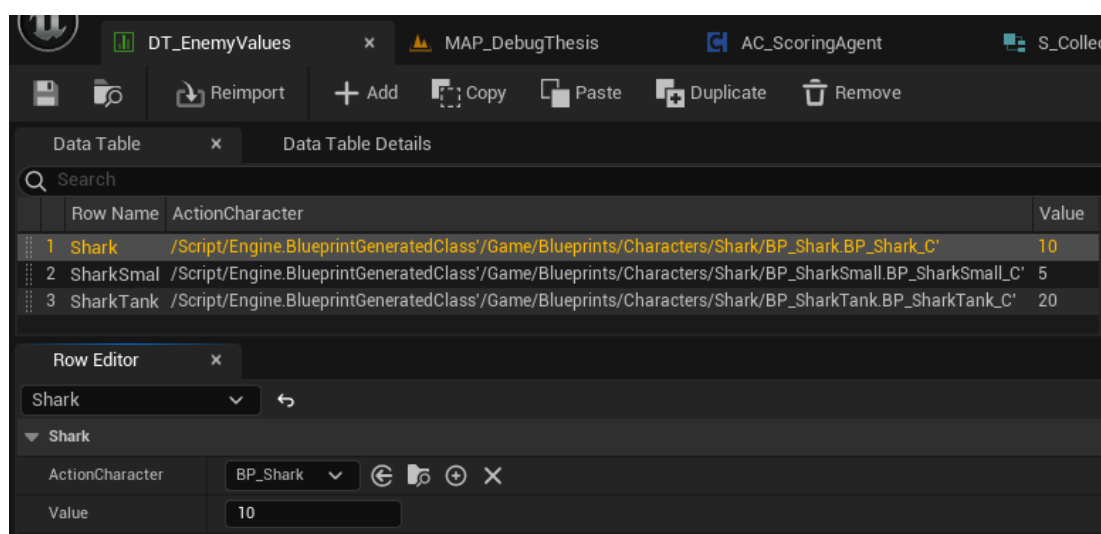
Z założenia, im większa jest wartość „`CurrentPointPool`”, tym większy będzie stopień trudności wygenerowanej sceny.

Wartość puli punktowej jest zwiększana na koniec każdego etapu za pomocą funkcji „`IncreasePointPool`” o wartość zmiennej `integer` „`PointIncreasePerLevel`” z różnicą odpowiednio dodanych lub odejmowanych punktów, zależnie od oceny wyniku gracza na aktualny etap. Wpływ oceny na pulę punktowo jest liczony osobno dla każdej klasy pokonanego przeciwnika w trakcie etapu.

Tabela 3.1. Struktura S_ActorValue

Nazwa zmiennej	Typ zmiennej	Opis
ActionCharacter	BP_ActionChar	Klasa aktora przeciwnika
Value	Integer	Wymagana wartość punktowa do wydania w celu wygenerowania instancji klasy ActionCharacter.

W celu edycji tabeli danych, wykorzystywany jest edytor widoczny na grafice poniżej (rys. 3.6.).



Rysunek 3.6. Edytor tabeli danych DT_EnemyValues

Źródło: Opracowanie własne

3.4. Implementacja proceduralnej generacji scenarii

Utworzony w ramach pracy prototyp posiada trzy warianty dostępnych map zawierających oznaczenia dostępnych miejsc do generacji przeciwników, postaci gracza oraz zakończenia poziomu. Wykorzystywane warianty zostały ukazane w grafice poniżej (rys. 3.7.).



Rysunek 3.7. Wariacje map w prototypie
Źródło: Opracowanie własne

Powyżej przedstawione mapy (rys. 3.7.) posiadają trzy typy znaczników z metadanymi wymaganymi do generacji sceny, ukazane w poniższej tabeli (tab. 3.2.):

Tabela 2.2. Znaczniki używane do generacji proceduralnej mapy

Znacznik	User Data Name	Opis
	EnemySpawn	Miejsce w którym może zostać wygenerowany przeciwnik. Mapa może zawierać dowolną ilość tego znacznika. Większa ilość znaczników zwiększa maksymalny możliwy poziom trudności na mapie.
	PlayerStart	Miejsce z którego startuje gracz. Mapa powinna zawierać tylko jeden znacznik PlayerStart.

Znacznik	User Data Name	Opis
	LevelEnd	Miejsce do którego gracz musi dotrzeć po pokonaniu wszystkich przeciwników. Mapa może zawierać dowolną ilość tego znacznika.

Podczas generowania nowej sceny, algorytm wpierw wybiera losowo jedną z dostępnych map, która jest w stanie użyć całej dostępnej puli punktów. Ilość punktów jaką mapa może maksymalnie użyć jest liczona według wzoru poniżej:

$$MaxPoints_{Map} = Count(EnemySpawn) \cdot Max(EnemyCost) \quad (1)$$

gdzie:

$MaxPoints_{Map}$ – Maksymalna ilość punktów do wydania na mapie,

$Count(EnemySpawn)$ – Ilość markerów z User Data Name „EnemySpawn” znajdujących się na mapie,

$Max(EnemyCost)$ – Największy koszt punktowy z dostępnych przeciwników w tabeli danych DT_EnemyValues.

W przypadku, kiedy żadna mapa nie jest w stanie użyć całej dostępnej puli punktów, wybierany jest losowy wariant ze wszystkich map o największej maksymalnej ilości punktów do wydania.

Po załadowaniu sceny z wylosowaną mapą, dalszą generacją sceny zajmuje się klasa „GM_Thesis” typu Game Mode, której typ jest odpowiedzialny w projektach tworzonych w Unreal Engine za deklarację zasad oraz działalność trybu gry.

Podczas uzupełniania mapy, postać gracza jest generowana na pierwszym znalezionym znaczniku z User Data Name „PlayerStart” oraz na każdym znaczniku z User Data Name „LevelEnd” jest generowany aktor „BP_LevelEnd”, będący celem końcowym pozwalającym graczowi przejść do następnego etapu. Przeciwnicy którzy będą na scenie, są uzupełniani losowo poprzez generację proceduralną według algorytmu przedstawionego poniżej.

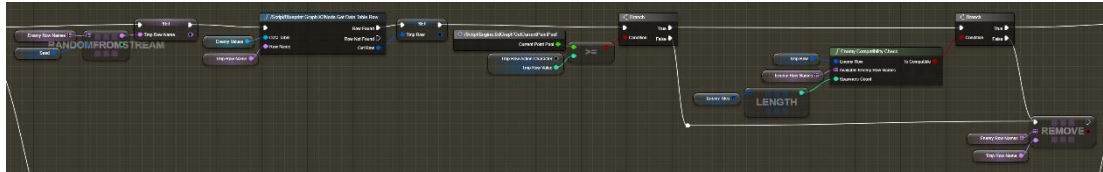
Po znalezieniu wszystkich znaczników z User Data Name „EnemySpawn” oraz załadowaniu nazw rzędów z tabeli danych „DT_EnemyValues” zawierającej dostępnych przeciwników oraz ich koszty generacji do tymczasowej puli generacji, wykonywana jest następująca pętla instrukcji, aż nie skończą się dostępne punkty w puli, znaczniki „EnemySpawn” lub dostępni przeciwnicy:

1. Załadowywane są informacje o klasie i koszcie punktowym losowego przeciwnika z dostępnej puli i algorytm weryfikuje czy koszt punktowy jest

mniejszy bądź równy ilości pozostałych punktów do wydania oraz czy wygenerowanie wylosowanego przeciwnika pozwoli na zużycie reszty dostępnej puli punktowej.

Jeżeli wylosowany typ przeciwnika nie spełnia tych warunków, zostaje on usunięty z dostępnej puli przeciwników oraz kończy aktualną iterację pętli.

Implementacja powyższej funkcjonalności została ukazana w grafice poniżej (rys. 3.8.):



Rysunek 3.8. Fragment funkcji SpawnEnemies w klasie GM_Thesis losująca klasę przeciwnika i weryfikująca czy klasa może zostać wygenerowana

Źródło: Opracowanie własne

Weryfikacja czy generacja wybranego przeciwnika pozwoli na dalsze wykorzystanie puli punktowej jest wykonywana przez funkcję „EnemyCompatibilityCheck”, która zlicza ilość typów przeciwników z dostępnej puli spełniających warunek ze wzoru poniżej. Wylosowany przeciwnik może zostać wygenerowany, jeżeli ilość typów spełniających warunek jest większa od 0.

$$EnemyCost_i + (EnemyCost_j \cdot (Count(RemainingEnemySpawn) - 1)) \leq RemainingPoints \quad (2)$$

gdzie:

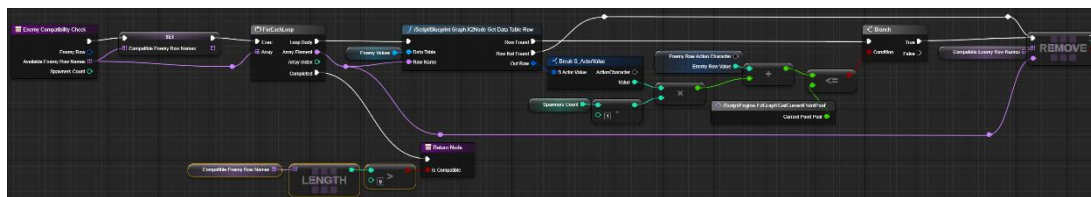
$EnemyCost_i$ – Koszt punktowy wylosowanego przeciwnika,

$EnemyCost_j$ – Koszt punktowy sprawdzanego typu przeciwnika z puli dostępnych przeciwników,

$Count(RemainingEnemySpawn)$ – Ilość dostępnych markerów z User Data Name „EnemySpawn” znajdujących się na mapie,

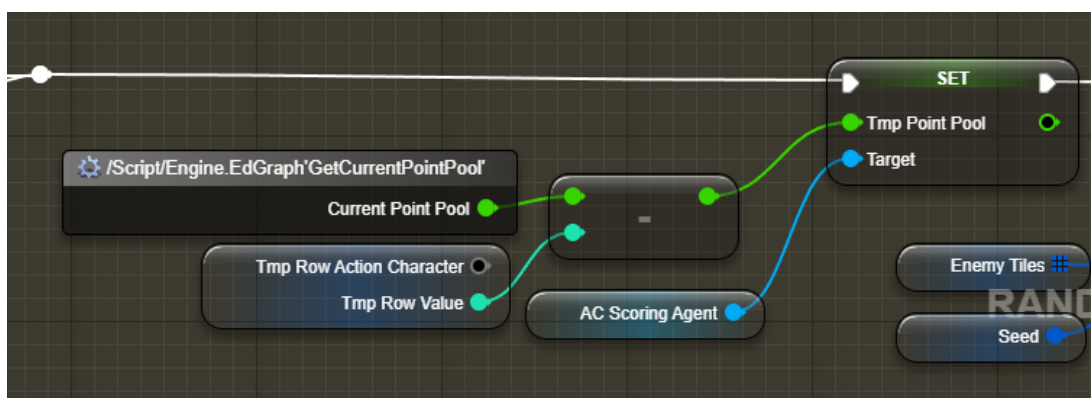
$RemainingPoints$ – Pozostała ilość punktów w puli do wydania.

Implementacja powyższej funkcji została ukazana w grafice poniżej (rys. 3.9.).



Rysunek 3.9. Funkcja EnemyCompatibilityCheck w klasie GM_Thesis
Źródło: Opracowanie własne

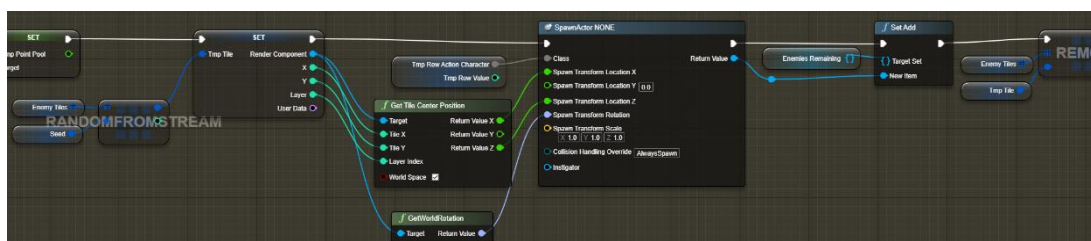
2. Po pomyślnej weryfikacji, czy dana klasa może zostać wygenerowana, odejmowany jest jej koszt punktowy od pozostałej puli punktów poprzez kod zaprezentowany w poniższym rysunku (rys. 3.10.),



Rysunek 3.10. Fragment funkcji SpawnEnemies w klasie GM_Thesis odpowiedzialny za aktualizację pozostałej puli punktowej w klasie AC_ScoringAgent

Źródło: Opracowanie własne

3. Po odjęciu kosztu, losowany jest znacznik „EnemySpawn” i generowana jest instancja wylosowanej klasy przeciwnika w koordynatach znacznika na scenie, następnie wylosowany znacznik jest usuwany z puli generacji, za pomocą kodu przedstawionego poniżej (rys. 3.11.).

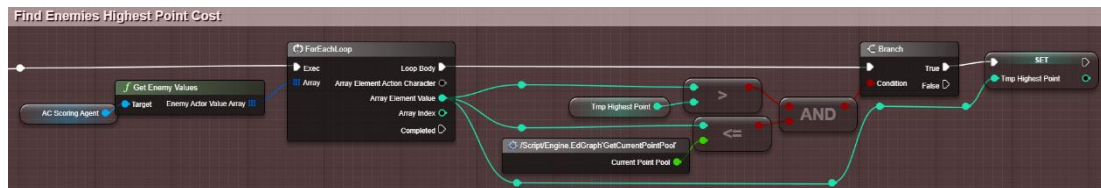


Rysunek 3.11. Fragment funkcji SpawnEnemies w klasie GM_Thesis odpowiedzialny za losowanie znacznika "EnemySpawn" i generację instancji wylosowanej klasy przeciwnika

Źródło: Opracowanie własne

W momencie przerwania powyższej pętli, jeżeli punkty do wydania nie zostały w pełni wykorzystane oraz w puli generacji znajdują się nieużyte znaczniki „EnemySpawn”, funkcja wykonuje poniższe instrukcje:

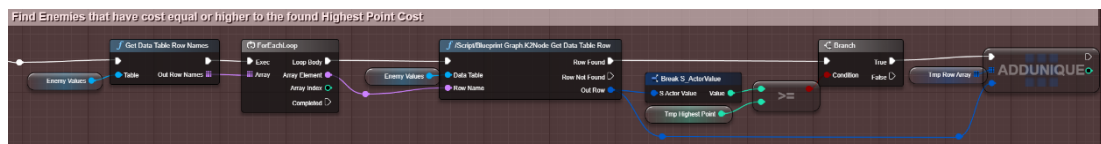
1. Znajduje najwyższy koszt punktowy wśród wszystkich klas przeciwników z tabeli danych „DT_EnemyValues” którego koszt jest mniejszy bądź równy pozostałej ilości punktów do wydania, za pomocą poniższego kodu ukazanego poniżej (rys. 3.12.),



Rysunek 3.12. Fragment funkcji SpawnEnemies w klasie GM_Thesis odpowiedzialna za znalezienie najwyższego możliwego kosztu punktowego do wydania z wszystkich klas przeciwników

Źródło: Opracowanie własne

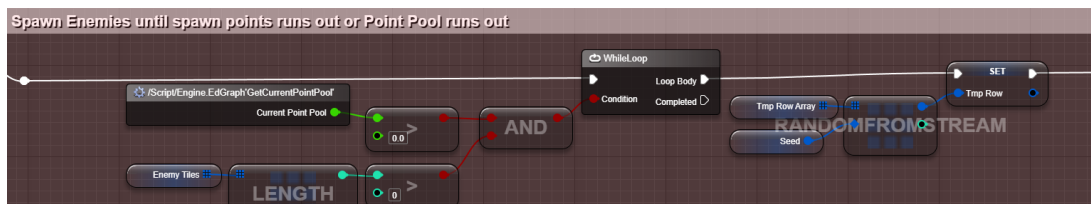
2. Znajduje oraz zapisuje do puli generacji wszystkich dostępnych przeciwników z tabeli danych „GM_Thesis” z kosztem punktowym równemu lub większemu od znalezionego najwyższego kosztu punktowego z poprzedniego kroku, wykorzystując fragment kodu przedstawiony poniżej (rys. 3.13.),



Rysunek 3.13. Fragment funkcji SpawnEnemies w klasie GM_Thesis odpowiedzialny za znalezienie klas przeciwników o koszcie równym lub wyższym znalezionemu najwyższemu kosztu punktowego

Źródło: Opracowanie własne

3. Funkcja wykonuje pętlę losującą klasę przeciwnika z dostępnej puli generacji oraz generując jej instancję na losowym znaczniku „EnemySpawn” poprzez odwołanie się do fragmentu funkcji przedstawionego w rysunku 3.11, do momentu aż pozostała pula punktowa będzie mniejsza lub równa 0, bądź skończą się dostępne znaczniki „EnemySpawn”. Obsługa powyższego kroku jest wykonywana przez kod przedstawiony w poniższym rysunku (rys. 3.14.).



Rysunek 3.14. Fragment funkcji SpawnEnemies w klasie GM_Thesis wykonująca pętlę losowania i generacji klas przeciwników z puli

Źródło: Opracowanie własne

W momencie zakończenia funkcji generowania przeciwników, jeżeli żadne znaczniki „EnemySpawn” nie będą dostępne, lecz nie zostaną wykorzystane wszystkie punkty z puli do wydania, oznacza to, że została wygenerowana scena o największym możliwym poziomie trudności jaki może zostać wygenerowany w aktualnym projekcie.

3.5. Konfiguracja i kryteria metody oceniania gracza

W ramach tworzonego algorytmu będącego celem pracy, algorytm został skonfigurowany z założeniami przedstawionymi poniżej.

Typ danych kontrolnych

Jako typ wykorzystywanych danych kontrolnych została wybrana ocena na bazie aktualnego przebiegu.

Wyniki gracza są zbierane osobno dla każdej klasy przeciwnika które znajdowały się na scenie i porównywane są z odpowiadającej klasie danymi zebranych z poprzednich etapów.

Kalibracja

Zmienna określająca ilość wymaganych danych kontrolnych aby można było dokonać oceny na bazie danych z aktualnego poziomu.

Przechowywana jest w zmiennej „CalibrationRequirement” w klasie „AC_ScoringAgent” z domyślną wartością ‘3’. Wartość tej zmiennej nie może wynosić poniżej 0.

Im większa wartość wymogu kalibracji, tym większa precyzja oceny wyników gracza kosztem czasu wymaganego, zanim wynik gracza zacznie być oceniany, w wyniku czego będzie miał wpływ na poziom trudności.

Próg błędu

Zmienna określająca stopień błędu w zakresie od 0% do 100% którego wynik gracza może odbiegać od średniej wynikającej z danych kontrolnych bez wpływu na poziom trudności. Przechowywana w zmiennej typu float „DeviationRange” w klasie „AC_ScoringAgent” z domyślną wartością ‘0.05’ równoważną z wartością 5%.

Im mniejszy dozwolony próg błędu, tym mniejsza różnica wyniku gracza od oczekiwanej średniej, jaka jest wymagana aby ocena miała wpływ na poziom trudności.

Startowa pula punktów

Zmienna określająca ilość dostępnych punktów w puli na start każdej rozgrywki. Przechowywana w zmiennej „StartingPointPool” w klasie „AC_ScoringAgent” z domyślną wartością ‘50’.

Im większa startowa pula punktów, tym większy poziom trudności pierwszej wygenerowanej sceny.

Zwiększenie puli punktów

Zmienna określająca wartość, o jaką pula punktowa będzie zwiększana z każdym kolejnym etapem. Przechowywana w zmiennej „PointIncreasePerLevel” w klasie „AC_ScoringAgent” z domyślną wartością ‘10’.

Wartość zwiększania puli punktów wpływa na zmiany puli punktowej wynikającej z oceny gracza.

Im większa wartość zmiennej, tym szybciej będzie się zwiększał poziom trudności co każdą scenę.

3.5.1. Parametry oceny

Parametry przedstawione w tabeli poniżej (tab. 3.3.) są obserwowane i wykorzystywane jako dane kontrolne do oceny wyników gracza. Dane kontrolne są przechowywane w osobnym zestawie danych odpowiadających każdej klasie przeciwnika, który był wygenerowany w rozgrywce.

Tabela 3.3. Parametry wykorzystywane do oceny gracza w prototypie tworzonego w ramach pracy

Parametr	Kryteria oceny	Opis
Otrzymane obrażenia	Im mniejsza ilość otrzymanych obrażeń tym lepsza ocena gracza.	Dla każdego przeciwnika zliczana jest ilość zadanych obrażeń w stronę gracza. Z założenia, im większa ilość obrażeń, którą otrzymuje gracz od danego typu przeciwnika, tym trudniejszy jest ten przeciwnik dla gracza.

Parametr	Kryteria oceny	Opis
Czas trwania walki	Im krótszy czas trwania walki, tym lepsza ocena gracza.	W momencie rozpoczęcia walki, deklarowaną przez otrzymanie obrażeń od gracza bądź wejście gracza w zasięg wykrywania przeciwnika uruchamia stoper, który zatrzymuje się w momencie pokonania przeciwnika. Z założenia, im krótszy czas trwania walki, tym lepiej graczowi poszło przeciwko danemu przeciwnikowi.

W zależności od potrzeb oraz celu wykorzystania danych kontrolnych, może być zadeklarowana dowolna ilość parametrów dowolnego typu. Wybrane parametry mogą również wpływać na osobne elementy w projekcie.

3.6. Implementacja oceny umiejętności gracza

Oceną wyników gracza zajmuje się klasa „AC_ScoringAgent”.

Dane zbierane do oceny gracza które otrzymuje powyższa klasa są przechowywane odpowiednio w tablicach struktur „S_CollectedCombatData”:

- „AllCombatData” – przechowywujący dane zebrane z poprzednich scen,
- „CollectedCombatData” – przechowywujący dane zebrane z aktualnej sceny, podczas przejścia na kolejny etap, dane z tej tablicy są dodawane do „AllCombatData”.

W tabelach poniżej (tab. 3.4., tab. 3.5.) są przedstawione struktury wykorzystywane do przechowywania danych kontrolnych:

Tabela 3.4. Struktura S_CollectedCombatData

Nazwa zmiennej	Typ zmiennej	Opis
ActionCharacter	BP_ActionChar	Klasa aktora przeciwnika
CombatDataArray	Tablica (S_CombatData)	Tablica struktur przechowujących dane obserwowanych parametrów oceny z osobnych instancji klas przeciwnika

Tabela 3.5. Struktura S_CombatData

Nazwa zmiennej	Typ zmiennej	Opis
DamageReceived	Float	Ilość obrażeń które otrzymał gracz od instancji obserwowanej klasy przeciwnika

Nazwa zmiennej	Typ zmiennej	Opis
TimeToKill	Float	Czas trwania walki z instancją obserwowanej klasy przeciwnika, od momentu pierwszego otrzymania obrażeń od gracza i/lub momentu wejścia gracza w zasięg wykrywania przeciwnika, do momentu pokonania przeciwnika

Na koniec każdego etapu, wywoływana jest funkcja „IncreasePointPool” w klasie „AC_ScoringAgent”, modyfikująca pulę punktów według wzoru poniżej:

$$PointPool = PointPool + PointIncrease + \sum_{i=0}^n PerformanceAddition_i \quad (3)$$

gdzie:

PointPool – Wartość zmiennej „CurrentPointPool”,

PointIncrease – Wartość zmiennej „PointIncreasePerLevel”,

n – Ilość elementów w tablicy „CollectedCombatData”,

PerformanceAddition – Ilość punktów dodawanych do puli wynikających z oceny gracza dla parametrów otrzymanych obrażeń oraz czasu trwania walki od klasy przeciwnika z tablicy „CollectedCombatData” o indeksie *i*.

W celu obliczenia zmiennej *PerformanceAddition* z powyższego wzoru, wykorzystuje się funkcję z poniższego wzoru, wykonywanego przez funkcję „CalculateFloatPerformanceAddition” dla obu parametrów oceny:

$$f(a, b, c, d) = \begin{cases} 0 & \text{jeżeli } Count(d) < CalibrationRequirement \\ 0 & \text{jeżeli } Avg(c) \in (Avg(d) - ((Avg(d) - Min(d)) \cdot b); Avg(d) + ((Max(d) - Avg(d)) \cdot d)) \\ \left| a - \left(a \cdot \frac{Avg(d)}{Avg(c)} \right) \right| & \text{jeżeli } \left| a - \left(a \cdot \frac{Avg(d)}{Avg(c)} \right) \right| \geq a \\ - \left| a - \left(a \cdot \frac{Avg(d)}{Avg(c)} \right) \right| & \text{jeżeli } \left| a - \left(a \cdot \frac{Avg(d)}{Avg(c)} \right) \right| < a \end{cases} \quad (4)$$

gdzie:

a – Zwiększenie puli punktów,

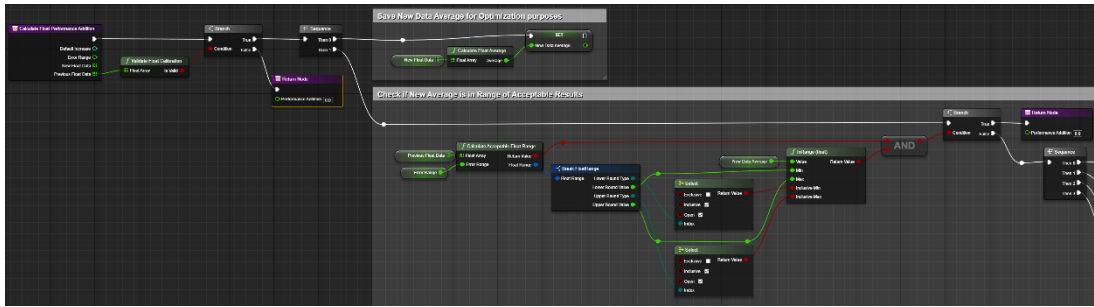
b – Próg błędu,

c – Tablica float zawierające dane z aktualnego etapu,

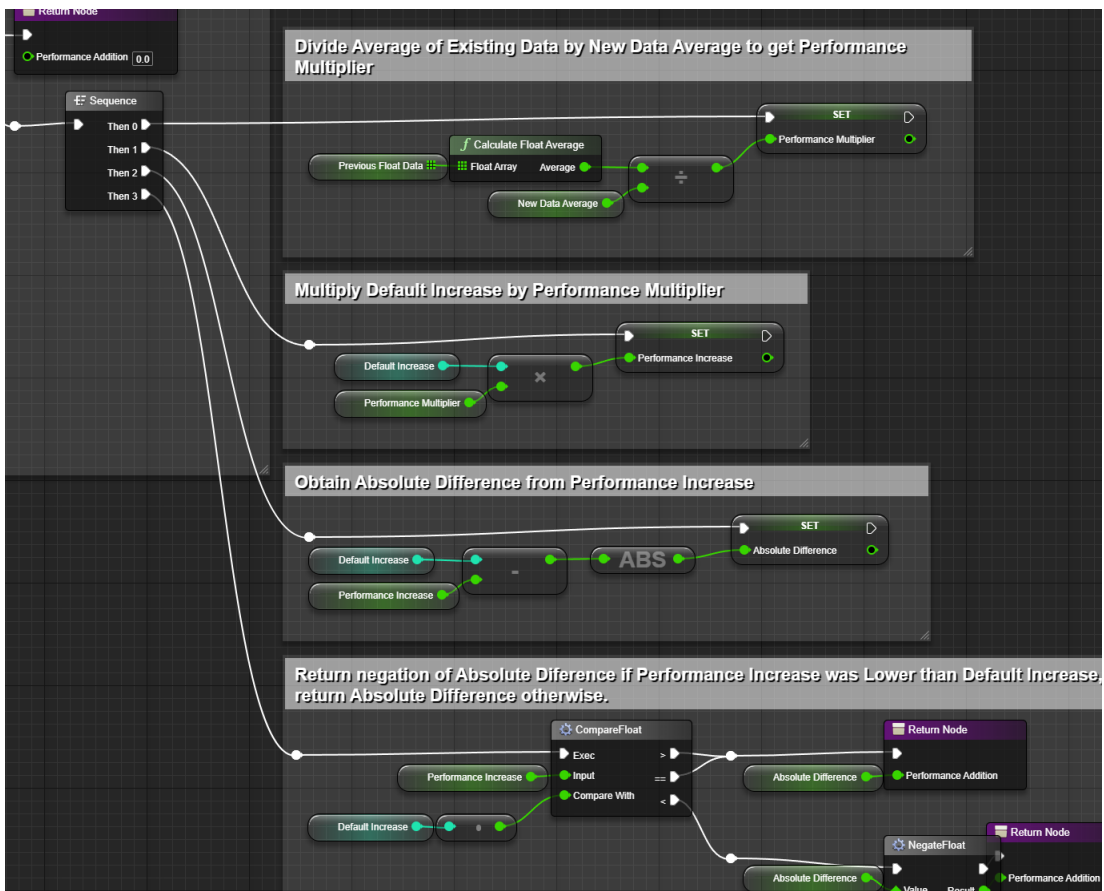
d – Tablica float zawierająca dane z poprzednich etapów,

CalibrationRequirement – Wartość zmiennej „CalibrationRequirement”.

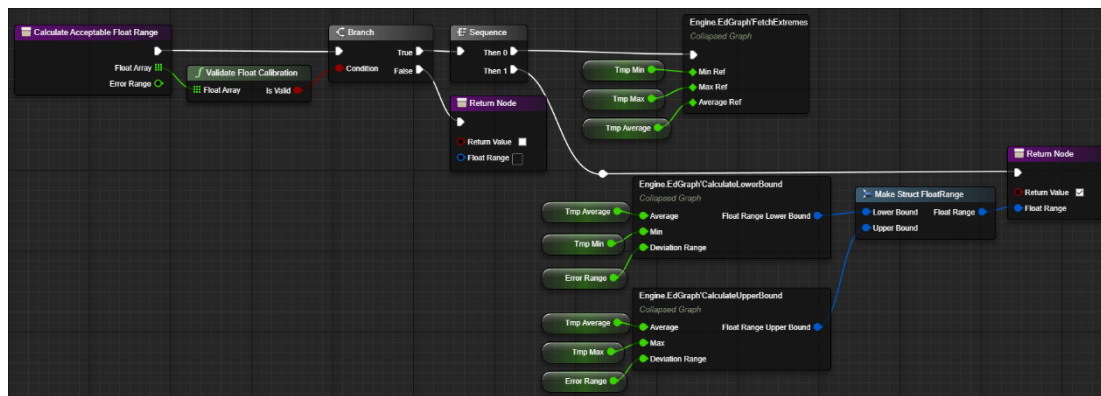
Implementacja funkcji “CalculateFloatPerformanceAddition” oraz wykorzystywanym w nich funkcji „CalculateAcceptableFloatRange” została zaprezentowana na rysunkach poniżej (rys. 3.15., rys. 3.16., rys. 3.17., rys. 3.18., rys. 3.19.).



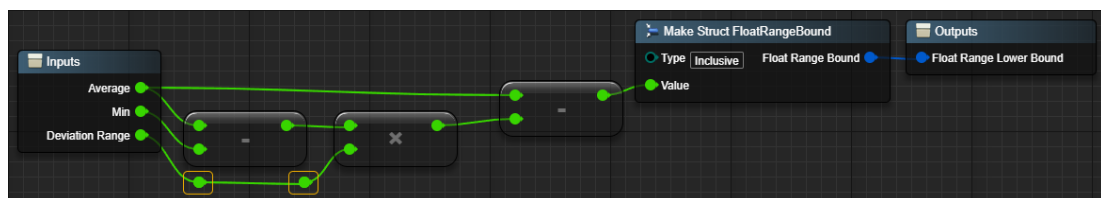
Rysunek 3.15. Lewa część funkcji "CalculateFloatPerformanceAddition" w klasie "AC_ScoringAgent"
Źródło: Opracowanie własne



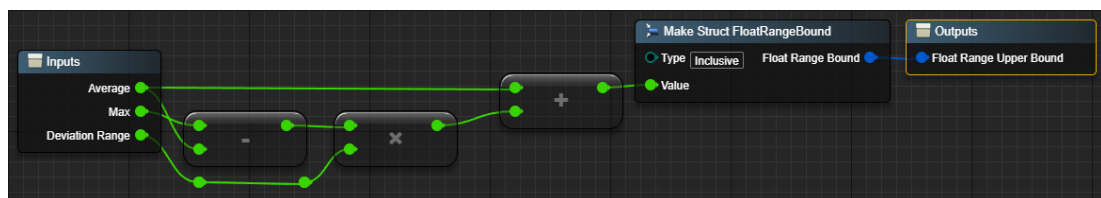
Rysunek 3.16. Prawa część funkcji "CalculateFloatPerformanceAddition" w klasie "AC_ScoringAgent"
Źródło: Opracowanie własne



Rysunek 3.17. Funkcja "CalculateAcceptableFloatRange" w klasie "AC_ScoringAgent" wykorzystywana w funkcji "CalculateFloatPerformanceAddition"
Źródło: Opracowanie własne



Rysunek 3.18. Podgraf "CalculateLowerBound" funkcji "CalculateAcceptableFloatRange"
Źródło: Opracowanie własne



Rysunek 3.19. Podgraf "CalculateUpperBound" funkcji "CalculateAcceptableFloatRange"
Źródło: Opracowanie własne

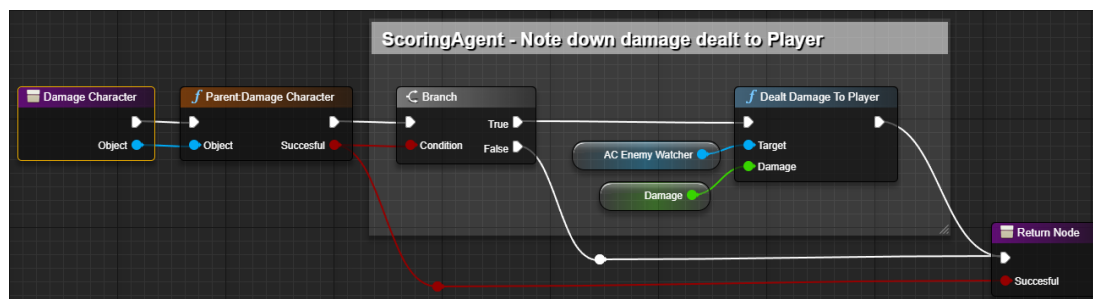
3.7. Implementacja obserwatora przeciwników

Aby klasa „AC_ScoringAgent” mogła funkcjonować poprawnie, potrzebny jest sposób na zbieranie danych wymaganych do oceny gracza. Tą rolę spełnia obserwator klasy „AC_EnemyWatcher”. Klasa obserwatora jest implementowana w klasach przeciwnika, które są częścią gry, poprzez dodanie instancji klasy jako komponent klasy przeciwnika oraz zintegrowanie istniejących funkcji aby „AC_EnemyWatcher” odpowiednio reagował na akcje oraz na stan przeciwnika.

Obserwacja otrzymywanych obrażeń

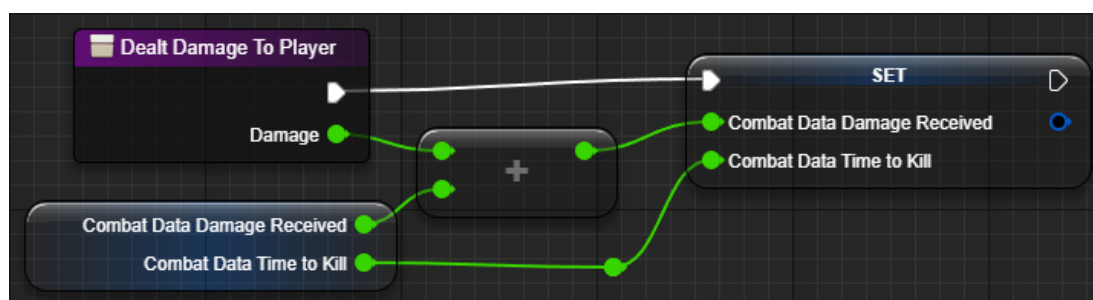
W celu zbierania danych o zadanych obrażeniach graczowi przez obserwowanego przeciwnika, funkcja „DamageCharacter” w klasie „BP_Shark” została rozszerzona

o wywołanie funkcji „DealtDamageToPlayer” w klasie „AC_EnemyWatcher”, w celu przekazania obserwatorowi informacji o ilości zadanych obrażeń graczowi, poprzez implementację ukazaną poniżej (rys. 3.20.).



Rysunek 3.20. Override funkcji "DamageCharacter" w klasie "BP_Shark"
Źródło: Opracowanie własne

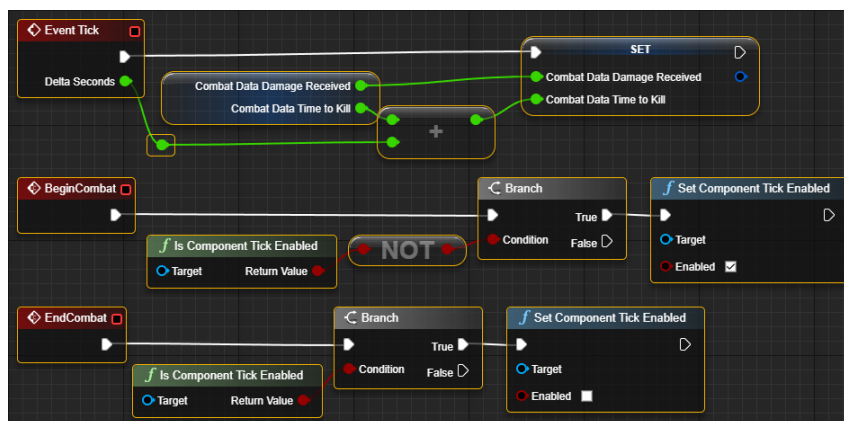
Otrzymana wartość obrażeń jest dodawana do struktury „CombatData” typu „S_CombatData”, przechowująca ilość otrzymywanych obrażeń oraz czas trwania walki po wywołaniu funkcji przedstawionej poniżej (rys. 3.21.).



Rysunek 3.21. Funkcja "DealtDamageToPlayer" w klasie "AC_EnemyWatcher"
Źródło: Opracowanie własne

Obserwacja czasu trwania walki

W celu zliczania czasu walki przeciwko danemu przeciwnikowi, klasa „AC_EnemyWatcher” dodaje do struktury „CombatData” co każdą klatkę animacji czas który minął od ostatniej wyrenderowanej klatki animacji poprzez event „Tick”.



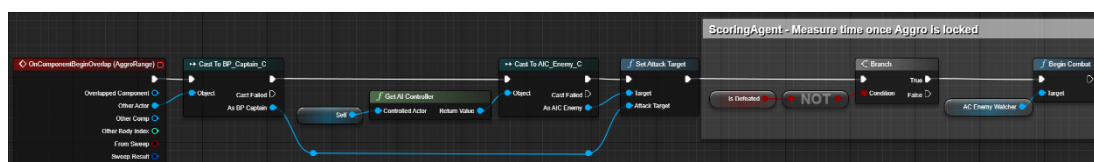
Rysunek 3.22. Eventy odpowiedzialne za zliczanie czasu trwania walki w klasie "AC_EnemyWatcher"

Źródło: Opracowanie własne

Aby event „Tick” był aktywny, funkcjonalność Tick w klasie musi być aktywna. Domyślnie jest ona wyłączona, aby nie był zliczany czas trwania walki od momentu wygenerowania się obserwowanego przeciwnika. Na aktywację oraz dezaktywację funkcjonalności Tick pozwalają eventy „BeginCombat” oraz „EndCombat” widoczne na grafice powyżej (rys. 3.22.), które są wywoływane poprzez obserwowanego przeciwnika.

Event „BeginCombat” jest wywoływany na dwa sposoby oznaczające rozpoczęcie walki:

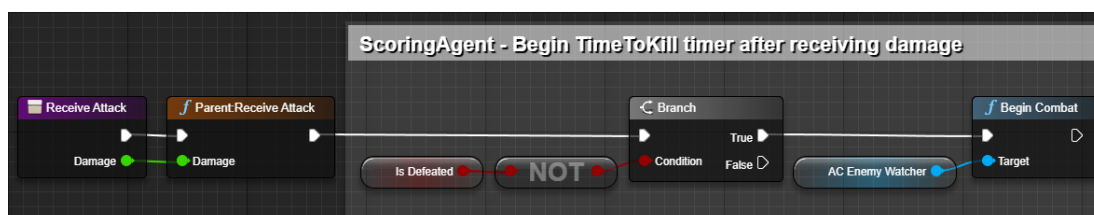
1. Gracz wejdzie w zasięg wykrycia obserwowanego przeciwnika, poprzez rozszerzenie kodu ukazane w grafice poniżej (rys. 3.23.),



Rysunek 3.23. Event OnComponentBeginOverlap(AggroRange) w klasie "BP_Shark"

Źródło: Opracowanie własne

2. Obserwowany przeciwnik otrzyma obrażenia od gracza, za pomocą rozszerzenia funkcji „ReceiveAttack” ukazaną poniżej (rys. 3.24.).



Rysunek 3.24. Override funkcji "ReceiveAttack" w klasie "BP_Shark"

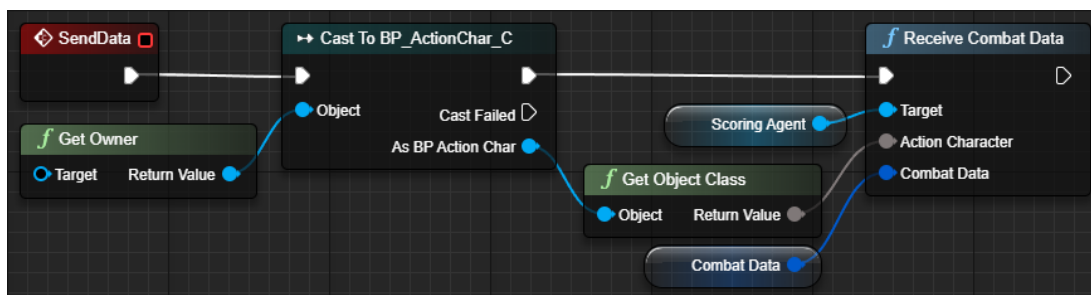
Źródło: Opracowanie własne

Przekazywanie zebranych danych

W momencie kiedy gracz pokona obserwowanego przeciwnika, wywoływane są eventy „EndCombat” aby zatrzymać naliczanie czasu trwania walki oraz „SendData” aby przesłać zebrane informacje o zadanych obrażeniach graczowi, w tym czasu trwania walki, wraz z klasą obserwowanego przeciwnika do klasy „AC_ScoringAgent” odpowiedzialnej za ocenę wyników gracza, jak ukazano na grafikach poniżej (rys. 3.25., rys. 3.26.).

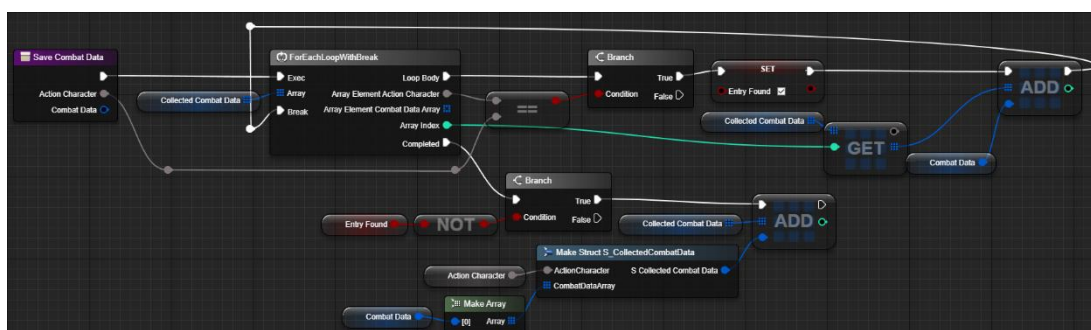


Rysunek 3.25. Fragment eventu "OnDefeated" w klasie "BP_Shark"
Źródło: Opracowanie własne



Rysunek 3.26. Event "SendData" w klasie "BP_EnemyWatcher"
Źródło: Opracowanie własne

Po otrzymaniu nowych danych, klasa „BP_ScoringAgent” zapisuje je w strukturze „CollectedCombatData” za pomocą funkcji „SaveCombatData” wykorzystując poniższy fragment kodu (rys. 3.27.).

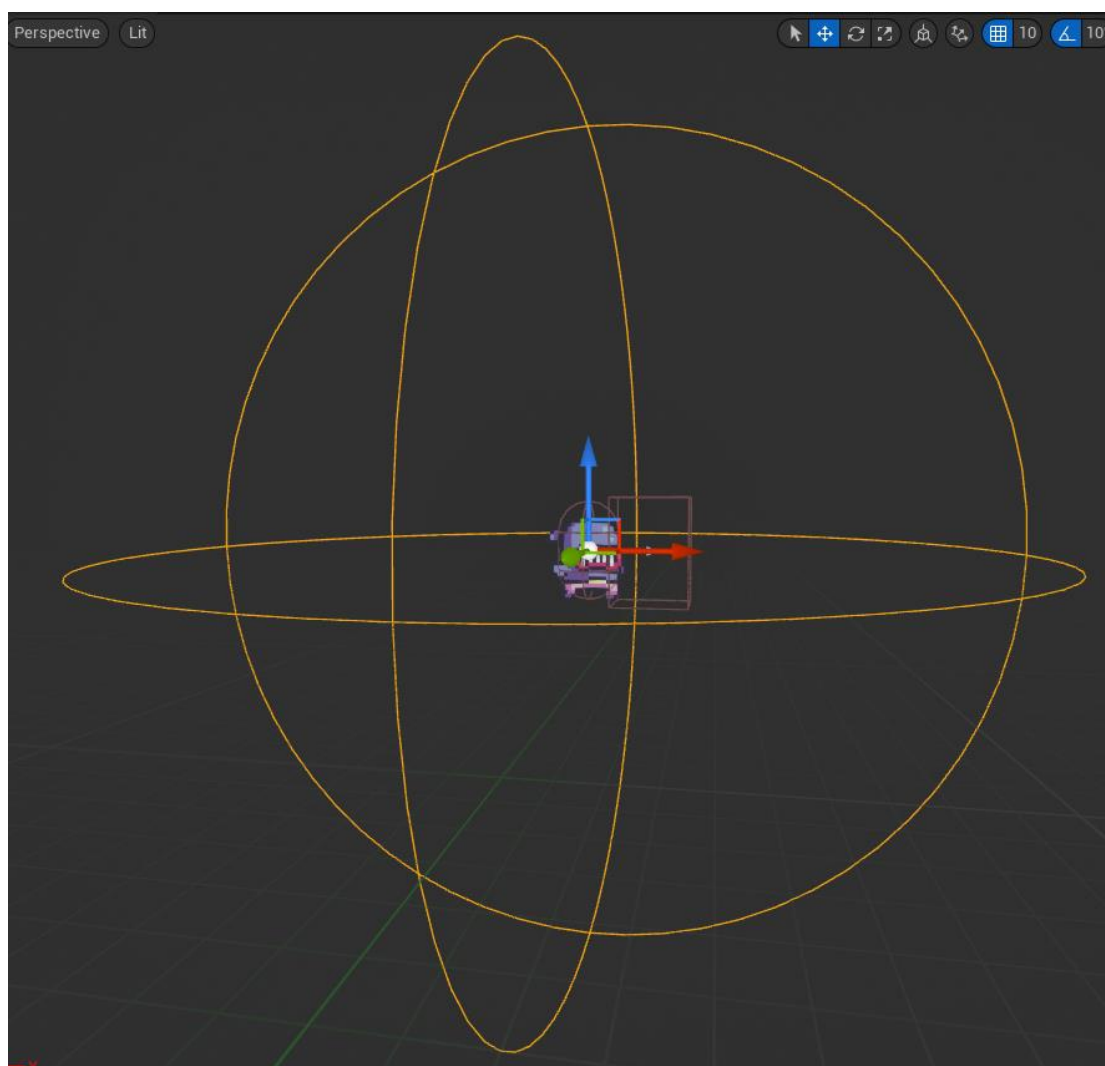


Rysunek 3.27. Funkcja "SaveCombatData" w klasie "AC_ScoringAgent"
Źródło: Opracowanie własne

Błąd implementacji obserwatora

W zademonstrowanej implementacji obserwacji czasu trwania walki, znajduje się błąd który został pozostawiony świadomie w celu prezentacji potencjalnych konsekwencji nieprawidłowego zaprogramowania obserwatorów, bądź nieodpowiednio zadeklarowanych parametrów oceny.

Zasięg wykrywania graczy u przeciwników został zaimplementowany poprzez sferę kolizyjną wokół modelu przeciwnika widoczną w grafice poniżej (rys. 3.28.). W momencie wejścia gracza w zasięg sfery, wywoływany jest event „OnComponentBeginOverlap(AggroRange)”.

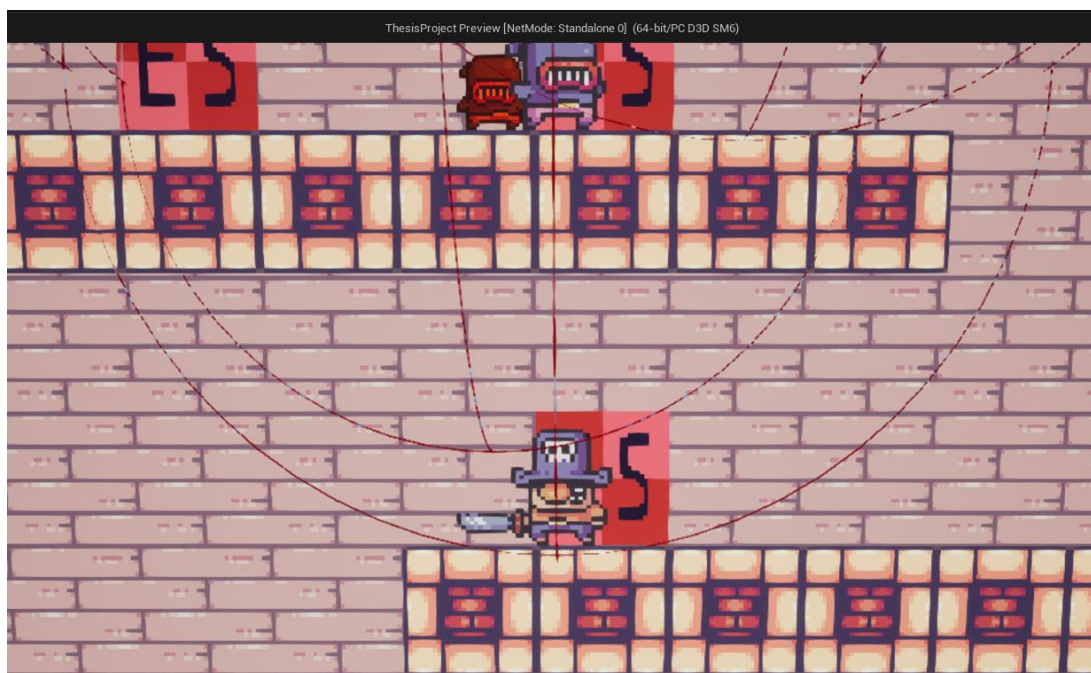


Rysunek 3.28. Podgląd klasy "BP_Shark" z hitboxem "AggroRange" (widocznym w formie sfer wokół modelu postaci)

Źródło: Opracowanie własne

Przez to, że aktywacja naliczania czasu walki przez obserwatora jest zintegrowana do eventu „OnComponentBeginOverlap(AggroRange)”, gdzie event nie posiada wykrycia czy

postać gracza nie jest zasłonięta przez inne elementy sceny, może pojawić się sytuacja, w której obserwator otrzyma informację o rozpoczęciu naliczania czasu walki, kiedy w rzeczywistości, nie byłoby możliwe aby obserwowany przeciwnik zobaczył postać gracza, bądź by miała możliwość poruszenia się w stronę lokacji gracza, jak ukazano w poniższym zdjęciu (rys. 3.29.).



Rysunek 3.29. Przykład niepożądaney aktywacji obserwatora - Postać gracza aktywowała wykrycie przez przeciwnika, aktywując naliczanie czasu walki przez obserwatora w momencie kiedy walka nie występuje

Źródło: Opracowanie własne

W przykładowej sytuacji przedstawionej powyżej, postać gracza weszła w zasięg wykrycia przeciwnika. Przez to, że obserwator zaczyna mierzyć czas trwania walki kiedy gracz wejdzie w zasięg wykrycia, gracz aktywuje w ten sposób naliczanie czasu trwania walki w sytuacji, która z teorii nie powinna się liczyć jako walka według założeń obserwowanych parametrów, przez fakt, że ani gracz ani obserwowany przeciwnik nie mają do siebie dostępu aby zainicjować faktyczną walkę.

W wyniku tego końcowy czas walki naliczany przez obserwatora staje się zawyżony oraz ma negatywny wpływ na ocenę wyników gracza, przez co algorytm oceniania nie będzie działał odpowiednio i zacznie zwracać błędne wyniki.

Podczas implementacji algorytmu będącego celem pracy, projektant gry oraz programista muszą dopilnować, aby nie tworzyły się sytuacje zwracające wyniki zwanymi „fałszywie dodatnie”.

Zakończenie

Celem niniejszej pracy było zaprojektowanie oraz zaimplementowanie rozwiązania do problemu projektowego, będącego ryzykiem powtarzalności i potencjalnego znużenia graczy z czasem trwania rozgrywki przy wykorzystywaniu generacji proceduralnej scenerii.

W pracy, został przedstawiony algorytm dynamicznej generacji scenerii, który jest rozwiązaniem powyższego problemu, poprzez dostosowywanie generacji scenerii do etapu gry oraz wyników gracza.

Algorytm został zaprojektowany i opisany w rozdziale drugim, ukazując najważniejsze elementy algorytmu, metodę działania, potencjalne możliwości konfiguracji oraz jego wykorzystanie. Całość została stworzona z myślą o uniwersalności oraz adaptacji, pozwalając na integrację z dowolnymi projektami i z możliwością modyfikacji w celu uzyskania pożądanych efektów.

W celu przetestowania utworzonego autorskiego algorytmu, w rozdziale trzecim algorytm został dostosowany i zaimplementowany w przykładowym projekcie platformowej gry wideo tworzonej na silniku Unreal Engine 5. Implementacja zakończyła się sukcesem. Została wykonana pozwalając na łatwą modyfikację, równocześnie zachowując integralność oryginalnego projektu gry, pozwalającą na dalszy rozwój projektu oraz nieinwazyjne modyfikacje samego algorytmu będącego celem pracy.

Zaprezentowana implementacja ukazała uniwersalność w modyfikacji oraz implementacji algorytmu oraz potencjalne zastosowania jakie przynosi wykorzystanie ukazanego algorytmu w ramach tworzonych projektów gier.

Streszczenie w języku polskim

Celem niniejszej pracy jest zaprojektowanie i implementacja algorytmu dynamicznej generacji scenarii dostosowującej się do etapu gry oraz wyników gracza, w celu rozwiązania problemu projektowego powtarzalności doświadczenia z czasem trwania rozgrywki, przy wykorzystywaniu generacji losowej, oraz stałego utrzymania zaangażowania gracza, w grach o nieskończonej długości rozgrywki.

Pierwszy rozdział przedstawia powszechne metody projektowania scenarii wraz z zastosowaniem, wadami i zaletami opisywanych metod oraz przykłady tytułów wykorzystujące przedstawiony sposób projektowania scen.

W drugim rozdziale zostaje przedstawiany autorski algorytm, opisując jego założenia, sposób działania oraz możliwości konfiguracji i jego zastosowanie.

Trzeci rozdział przedstawia przykładową implementację algorytmu z rozdziału drugiego w prototypie gry utworzonym na silniku Unreal Engine 5.

Streszczenie w języku angielskim

The goal of the present thesis is to design and implement a dynamic scene generation algorithm adapting to game phase and player performance, to solve a design problem of repetitive experience increasing with playtime, by the usage of random generation as well by upkeep of player engagement, in games with endless playtime.

First chapter presents popular scene design methods with their usage, cons and pros and provides examples of titles that utilize presented scene design methods.

Authorial algorithm is presented in the second chapter, detailing it's premise, the way it's working as well as configuration possibilities and it's usage.

Third chapter shows an example implementation of the algorithm from chapter two, that has been implemented in a game prototype made in Unreal Engine 5.

Bibliografia

1. Polewiak P., *Tworzyć gry*, Insignis Media, 2020
2. Gamma E., Helm R., Vlissides J., *Design Patterns of Reusable Object-Oriented Software*, Addison-Wesley Professional, 1994

Strony www

3. https://csgo.fandom.com/pl/wiki/Dust_II (dostęp 28.04.2025)
4. <https://store.steampowered.com/app/683320/GRIS/> (dostęp 28.04.2025)
5. Alan Zucconi, *The World Generation of Minecraft*, 2022, <https://www.alanzucconi.com/2022/06/05/minecraft-world-generation/> (dostęp 28.04.2025)
6. <https://steamcommunity.com/games/DeepRockGalactic/announcements/detail/4593196713081471259> (dostęp 28.04.2025)
7. Boris, *Dungeon Generation in Binding of Isaac*, 2020, <https://www.boristhebrave.com/2020/09/12/dungeon-generation-in-binding-of-isaac/> (dostęp 28.04.2025)
8. <https://tools.thecyclefrontier.wiki/map> (dostęp 28.04.2025)
9. Marek Winiarski, *Poziom trudności*, 2016, <https://mwin.pl/poziom-trudnosci/> (dostęp 28.04.2025)
10. <https://www.airforcemedicine.af.mil/News/Photos/igphoto/2000986255/> (dostęp 28.04.2025)
11. https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-programming-and-scripting?application_version=5.3 (dostęp 28.04.2025)
12. Cobra Code, *The Ultimate Unreal Engine 2D Game Development Course*, <https://www.udemy.com/course/unreal-2d-course/> (dostęp 28.04.2025)

Spis rysunków

Rysunek 1.1. Zrzut ekranu sceny "Dust II" z gry "Counter-Strike 2"	16
Rysunek 1.2. Zrzut ekranu z gry "GRIS"	17
Rysunek 1.3. Przykładowa generacja terenu o wielkości 2048 bloków w grze "Minecraft".	18
Rysunek 1.4. Wzór wygenerowanej jaskinii przed wygenerowaniem elementów graficznych w Deep Rock Galactic z poziomu edytora w silniku Unreal Engine 4	19
Rysunek 1.5. Wygenerowana jaskinia z rysunku 1.4.....	19
Rysunek 1.6. Przykładowa generacja mapy z gry "The Binding of Isaac: Rebirth". Każdy blok reprezentuje osobny wcześniej przygotowany fragment scenerii tworzący razem całą scenę.	21
Rysunek 1.7. Mapa "Bright Sands" z gry "The Cycle Frontier" z oznaczeniem kolorystycznym stref wpływających na wyniki generacji losowej.	22
Rysunek 2.1. Krzywa obrazująca doświadczenie gry przez gracza w projektowaniu gier	25
Rysunek 2.2. Zdjęcie przedstawiające prezentację gry Warhammer 40.000 w Operation Sci-Fi Con 2015, w Niemczech z 2015 roku.	26
Rysunek 2.3. Schemat opisujący metodę działania funkcjonalności generacji proceduralnej scenerii	29
Rysunek 2.4. Relacja obserwatora jeden do wielu.....	34
Rysunek 3.1. Edytor silnika Unreal Engine 5.3.2	37
Rysunek 3.2. Edytor Blueprint w edytorze Unreal Engine 5.3.2	37
Rysunek 3.3. Zrzut ekranu z gry używanej w projekcie	38
Rysunek 3.4. Lewa część funkcji IncreasePointPool w klasie AC_ScoringAgent.	39
Rysunek 3.5. Prawa część funkcji IncreasePointPool w klasie AC_ScoringAgent	39
Rysunek 3.6. Edytor tabeli danych DT_EnemyValues	40
Rysunek 3.7. Wariacje map w prototypie.....	41
Rysunek 3.8. Fragment funkcji SpawnEnemies w klasie GM_Thesis losująca klasę przeciwnika i weryfikująca czy klasa może zostać wygenerowana.....	43
Rysunek 3.9. Funkcja EnemyCompatibilityCheck w klasie GM_Thesis.....	44
Rysunek 3.10. Fragment funkcji SpawnEnemies w klasie GM_Thesis odpowiedzialny za aktualizację pozostałej puli punktowej w klasie AC_ScoringAgent.....	44
Rysunek 3.11. Fragment funkcji SpawnEnemies w klasie GM_Thesis odpowiedzialny za losowanie znacznika "EnemySpawn" i generację instancji wylosowanej klasy przeciwnika	44
Rysunek 3.12. Fragment funkcji SpawnEnemies w klasie GM_Thesis odpowiedzialna za znalezienie najwyższego możliwego kosztu punktowego do wydania z wszystkich klas przeciwników	45

Rysunek 3.13. Fragment funkcji SpawnEnemies w klasie GM_Thesis odpowiedzialny za znalezienie klas przeciwników o koszcie równym lub wyższy znalezionemu najwyższemu kosztu punktowego	45
Rysunek 3.14. Fragment funkcji SpawnEnemies w klasie GM_Thesis wykonująca pętle losowania i generacji klas przeciwników z puli	46
Rysunek 3.15. Lewa część funkcji "CalculateFloatPerformanceAddition" w klasie "AC_ScoringAgent"	50
Rysunek 3.16. Prawa część funkcji "CalculateFloatPerformanceAddition" w klasie "AC_ScoringAgent"	50
Rysunek 3.17. Funkcja "CalculateAcceptableFloatRange" w klasie "AC_ScoringAgent" wykorzystywana w funkcji "CalculateFloatPerformanceAddition"	51
Rysunek 3.18. Podgraf "CalculateLowerBound" funkcji "CalculateAcceptableFloatRange"	51
Rysunek 3.19. Podgraf "CalculateUpperBound" funkcji "CalculateAcceptableFloatRange"	51
Rysunek 3.20. Override funkcji "DamageCharacter" w klasie "BP_Shark"	52
Rysunek 3.21. Funkcja "DealtDamageToPlayer" w klasie "AC_EnemyWatcher"	52
Rysunek 3.22. Eventy odpowiedzialne za zliczanie czasu trwania walki w klasie "AC_EnemyWatcher"	53
Rysunek 3.23. Event OnComponentBeginOverlap(AggroRange) w klasie "BP_Shark"	53
Rysunek 3.24. Override funkcji "ReceiveAttack" w klasie "BP_Shark"	53
Rysunek 3.25. Fragment eventu "OnDefeated" w klasie "BP_Shark"	54
Rysunek 3.26. Event "SendData" w klasie "BP_EnemyWatcher"	54
Rysunek 3.27. Funkcja "SaveCombatData" w klasie "AC_ScoringAgent"	54
Rysunek 3.28. Podgląd klasy "BP_Shark" z hitboxem "AggroRange" (widocznym w formie sfer wokół modelu postaci)	55
Rysunek 3.29. Przykład niepożądaney aktywacji obserwatora - Postać gracza aktywowała wykrycie przez przeciwnika, aktywując naliczanie czasu walki przez obserwatora w momencie kiedy walka nie występuje	56

Spis tabel

Tabela 2.1. Typy danych kontrolnych dla algorytmu oceny umiejętności gracza	30
Tabela 3.1. Struktura S_ActorValue	40
Tabela 3.2. Znaczniki używane do generacji proceduralnej mapy	41
Tabela 3.3. Parametry wykorzystywane do oceny gracza w prototypie tworzonego w ramach pracy.....	47
Tabela 3.4. Struktura S_CollectedCombatData	48
Tabela 3.5. Struktura S_CombatData.....	48

Załącznik 1. Plyta CD z kodem programu